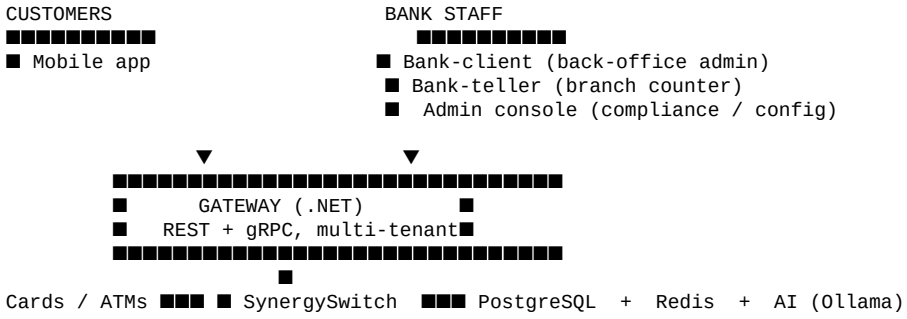
GoldBank — Executive Summary

> *Multi-tenant core banking platform with five front-ends, a payments switch, AI-powered KYC, and two flagship products: **physical-asset custody** (gold, silver, platinum) and **Ekub** community savings groups.*

What it is

A complete retail-banking platform built for a Southern African market. One gateway exposes everything; five clients consume it; one payments switch bridges the card-acceptance network.



What it does — at a glance

Capability	Status
Customer registration with KYC (national ID + selfie + proof of address)	■ live
Dual-currency accounts (ZWG + USD) per customer	■ live
Card issuance with virtual PAN	■ live
Send money (P2P), pay bills, cash-in/out at agents, QR pay, NFC tap	■ live
Mobile loans + repayment	■ live
Merchant onboarding + POS acceptance	■ live
Fraud monitoring with 5 rule types	■ live
Card switching (ISO 8583 + ISO 20022)	■ live
Physical-asset custody (gold / silver / platinum, valuation, lending against)	■ **flagship**
Ekub community savings groups (member voting, treasurer-confirmed loans)	■ **flagship**
AI features (OCR, fraud explanation, in-app chat) via local Qwen3-VL	■ live

Flagship product 1 — Asset Custody

Customers deposit physical precious-metal assets into **trusted vault facilities ("deposit houses")** that the bank vets. Every deposit is registered as an asset row with weight, purity, receipt number, and a chain of professional valuations. Three things make the product distinctive:

1 **Person-scoped, not account-scoped.** Gold doesn't belong to your USD or

ZWG account — it belongs to *you*. The data model reflects this through a `Customer` aggregate.

1 **Collateral lending.** Any asset can back a loan. The teller UI **blocks

asset withdrawal** when there's an open loan secured against it.

1. **Live spot pricing.** Daily metal prices feed into portfolio valuation;

members see their custody value updated in their mobile app.

Flagship product 2 — Ekub

A digital implementation of the rotating savings + lending tradition (called "Ekub" in some regions, "stokvel" / "chama" elsewhere). Members pool monthly contributions; any member can borrow against the pot subject to a vote.

Step	Who	What
1	Chairman	Creates a group (currency, monthly amount, interest rate)
2	Chairman / Secretary	Invites 2+ more members
3	Invitees	Accept invitations; group auto-activates at 3 members
4	All members	Contribute the agreed monthly amount
5	Treasurer	Confirms each contribution; the pot grows
6	Any member	Apply for a loan against the pot
7	Other members	Vote approve/reject (borrower can't vote)
8	Treasurer	Confirms disbursement once a majority approves
9	Borrower	Repays in monthly installments; interest grows the pot
10	All members	See their pro-rata share of the pot in real time

A configurable toggle lets groups choose to **charge no interest** on loans that stay within a borrower's own contributions — effectively letting members borrow their own money interest-free.

Stack summary

Layer	Technology
Server	.NET 10, Kestrel, EF Core 10, Wolverine, gRPC
Data	PostgreSQL 18, Redis 7
Switch	.NET 10, ISO 8583 TCP, gRPC
AI	Ollama + Qwen3-VL (local, no cloud dependency)
Mobile	Kotlin Multiplatform, Jetpack Compose, Koin
Web (admin + teller)	React 19, Vite 6, Material UI 6
Compliance console	Blazor Server
Orchestration	Docker Compose (Podman-compatible)

Operational shape

- **One container per service**, orchestrated by `docker-compose.yml` with

profile-based subsets (`core`, `ai`, `monitoring`).

- **Single Postgres instance**, two databases (`goldbank` for the bank,

`synergy_switch` for the switch).

- **Wolverine outbox** for at-least-once event delivery (notifications, audit).
- **Multi-tenant by design** — every aggregate carries a `tenant_id`. One

gateway can serve many brands.

Code layout

```
server/         .NET (Gateway, Core, Migrator, Notifications, Reporting)
switch/         SynergySwitch (ISO 8583 / 20022)
mobile/         Kotlin Multiplatform Android app
bank-client/    React back-office admin web
bank-teller/    React branch-counter web
admin/          Blazor compliance / system-config console
hsm/            Mock HSM for PIN crypto
docs/system/    Comprehensive technical documentation (you are here)
```

Maturity

Aspect	State
Customer-facing flows (mobile)	Production-ready feature surface; not yet load-tested
Teller flows (cash + asset deposit/withdrawal)	Real JWT auth, role-gated, audit-logged
Back-office admin	Functional but logs in via dev-stub seed accounts — **needs real auth before prod**
Card switching	Functional spec + service skeleton; needs real Zimswitch certification
AI features	Working locally with Ollama; needs GPU hosting for prod
Tests	Sparse — patchy unit coverage on .NET; no tests on web or mobile yet
CI/CD	`Jenkinsfile` + `.gitlab-ci.yml` exist but aren't wired to a runner

The repository is **demo-complete** — every flow works end-to-end on a developer laptop with seeded data. Hardening for production (real admin auth, KYC document object storage, secret rotation, observability stack, load testing, certification of card networks) is the remaining work.

Read more

The full technical documentation lives under docs/system/:

- architecture.md — system map + communication patterns
- data-model.md — schema, multi-tenancy, key invariants
- server.md — every module + every endpoint
- switch.md — SynergySwitch deep-dive
- mobile.md — Android app structure + features
- bank-client.md — back-office admin web
- bank-teller.md — branch teller web
- operations.md — running, deploying, troubleshooting

GoldBank System Documentation

Living documentation of the GoldBank platform. Updated as features land; aimed at a developer joining the project who needs to understand the shape of the system, not at end-users.

How to read

Pick your entry point:

If you want to…	Start here
Understand the whole system in 10 minutes	architecture.md
Find an API endpoint or service	server.md
Understand the DB schema	data-model.md
Work on the customer-facing Android app	mobile.md
Work on the back-office admin UI	bank-client.md
Work on the branch-teller UI	bank-teller.md
Understand card switching / EFT	switch.md
Run the system locally / debug containers	operations.md

The system in one paragraph

GoldBank is a **multi-tenant, multi-currency core banking platform** with five front-ends: a customer **mobile app** (Android), a back-office **bank-client** admin web app, a branch-counter **bank-teller** web app, an admin **Blazor** console, and a **synergy-switch** that bridges POS terminals and national payment networks (Zimswitch/ISO 8583). A single **.NET gateway** sits in the middle exposing **gRPC** (mobile, switch) and **REST** (web admin, teller) APIs over the same module backbone. State lives in **PostgreSQL** (multi-tenant schemas) and **Redis** (sessions, OTPs, rate limits). Long-running work and cross-module side-effects flow through a **Wolverine** in-process message bus with an outbox table; SMS / push notifications are dispatched by a separate **Notifications** service. AI features (OCR, fraud explanations, chat) call a local **Ollama** server running Qwen3-VL.

Two products you should know about

Beyond standard banking (accounts, transactions, cards, loans, KYC), two product surfaces are worth calling out:

- **Asset Custody** — customers deposit physical gold/silver/platinum into

trusted vault facilities ("deposit houses"). Each deposit is registered as an `Asset`, valued periodically, and can be used as **collateral** on a loan. Custody is **person-scoped** (a `Customer` aggregate, not an `Account`) so a deposit doesn't pick a currency. See `modules/asset-custody` in server.md and the Asset Custody section of bank-client.md.

- **Ekub** — a community savings + lending product modelled on traditional

rotating savings clubs. Members pool monthly contributions, members may borrow against the pot subject to a member vote, and the treasurer confirms disbursement. Interest can optionally be charged only on the portion of a loan that exceeds a borrower's own contributions. See `modules/ekub` in server.md and the Ekub feature in mobile.md.

Code map (where does what live)

Path	What	Stack
`server/GoldBank.Gateway/`	REST + gRPC entry point	.NET 10 / Kestrel

Path	What	Stack
`server/GoldBank.Core/Modules/<Name>/`	Domain logic per module	.NET / EF Core
`server/GoldBank.Migrator/`	EF migrations + demo seeder + CLI jobs	.NET console
`server/GoldBank.Notifications/`	SMS + FCM push dispatcher	.NET / Wolverine
`server/GoldBank.Reporting/`	Operational reports	.NET / gRPC
`server/GoldBank.Protos/Protos/`	Proto definitions (gRPC contracts)	proto3
`switch/`	SynergySwitch (POS/ATM EFT switch)	.NET / gRPC + ISO 8583 TCP
`admin/GoldBank.Admin/`	Compliance / system-config console	Blazor Server
`bank-client/`	Back-office operations web app	React + Vite + MUI
`bank-teller/`	Branch teller / supervisor / vault web app	React + Vite + MUI
`mobile/`	Customer mobile app	Kotlin Multiplatform + Compose
`hsm/`	Mock HSM for PIN/card crypto	.NET
`nfc-test-app/`	Standalone NFC HCE diagnostic app	Kotlin / Android
`docs/`	Documentation (you are here)	Markdown
`scripts/`	One-off SQL seeds + PowerShell jobs	SQL / PowerShell
`docker-compose.yml`	Container orchestration	Compose v3

Conventions

- **Branches**: trunk-based-ish; `main` is the only long-lived branch on

`origin`. Feature work goes in topic branches off `main`, squash-merged.

- **Commits**: conventional-commits style (`feat:`, `fix:`, `chore:`, etc.)

with a one-line subject and a body when the diff is non-trivial.

- **Tests**: a `tests/` project tree exists for .NET; current coverage is

patchy. Mobile/web have no unit tests yet (acknowledged debt).

- **Tenancy**: every persisted entity carries a `tenant_id`. The default

tenant for local dev is the string `"goldbank"`. See data-model.md.

- **Money on the wire**: amounts are sent as **decimal strings** in proto

messages (`"50.00"`) and wrapped in a `Money { amount, currency }` value. Avoid floating-point in domain logic.

- **IDs**: every aggregate uses a `Guid` PK. Where short, human-typeable IDs

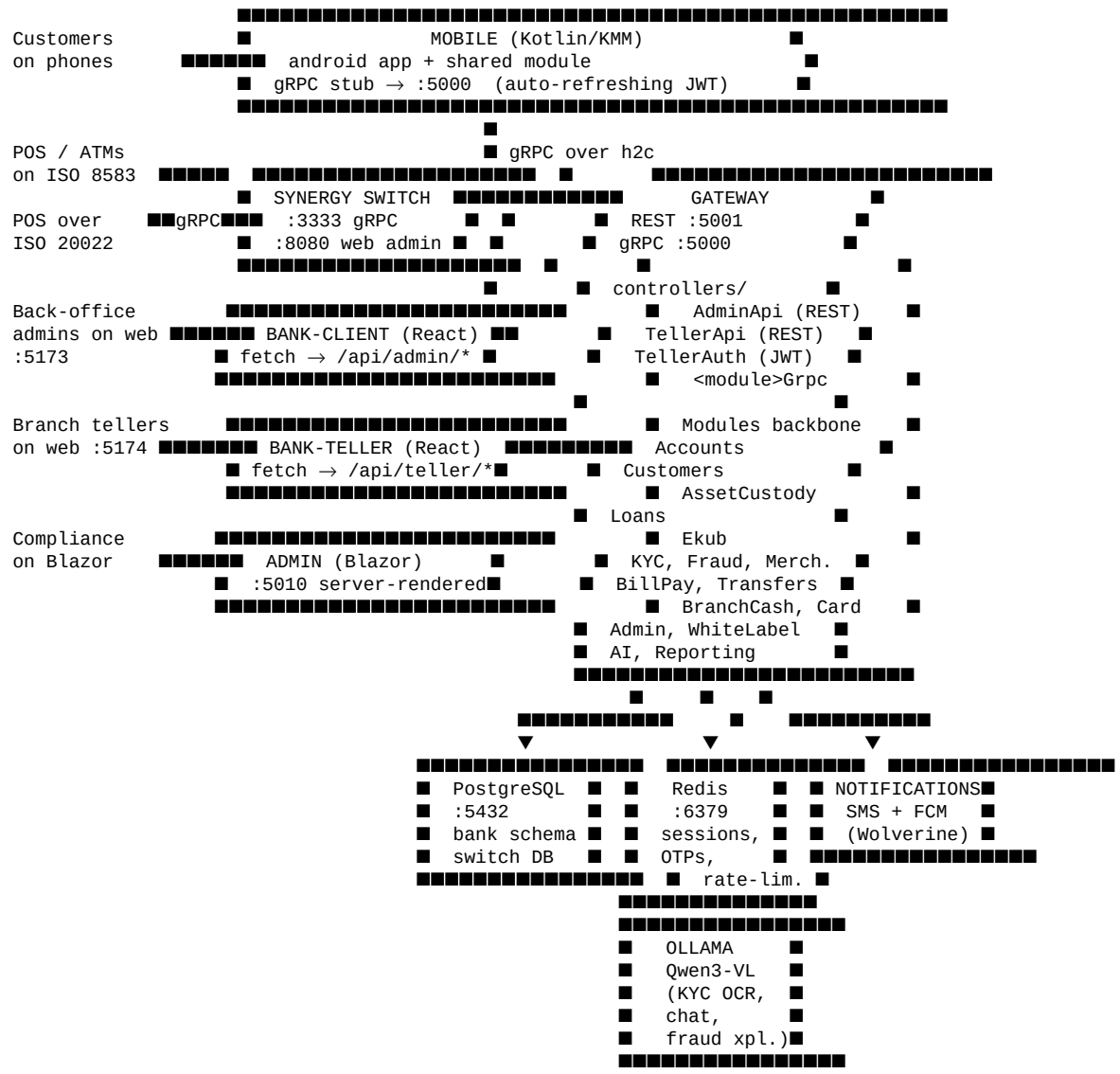
are useful (admin lists, teller screens) the gateway projects them via `ShortId(prefix, guid)` — e.g. `AST-000001`, `LOAN-043521`.

Historical context

A previous version of this codebase was named **UniBank** and the rename to GoldBank landed in commit 4be4dd7. Older docs at the top of `docs/` (with `unibank` in the filename) describe the state as of that rename and are kept as historical reference. Anything in this `docs/system/` subdirectory is the current source of truth.

Architecture

Component map



Communication patterns

1. Mobile → Gateway (gRPC over plaintext HTTP/2)

Mobile uses gRPC because:

- Bidirectional streaming for chat
- Server streaming for transaction lists
- Smaller payloads / less battery
- Strong typing across the wire

The gateway listens on **port 1111** inside the container, mapped to host **:5000**. Plaintext (no TLS) for dev; TLS terminated at the load balancer in prod. Each mobile call carries two headers:

- `authorization: Bearer <JWT>` — set by `AuthClientInterceptor`
- `x-tenant-id: goldbank` — set by the same interceptor

The JWT is refreshed transparently by `TokenRefresher.kt` which sits in front of every `grpcCall { … }` and re-issues the access token when it's within 60 seconds of expiry. Refresh is mutex-guarded with a re-entrancy marker so the refresh call itself doesn't deadlock.

2. Bank-client / bank-teller → Gateway (REST/JSON over HTTP/1.1)

The two React apps run on their own Vite dev servers (`:5173` for bank-client, `:5174` for bank-teller). They hit the gateway's **REST surface on port 1112** (host `:5001`). Bank-client uses session-based mock auth (currently hardcoded `SEED_ACCOUNTS` in the client); bank-teller uses **JWT** via `POST /api/teller/auth/login`. Both call `fetch('/api/<area>/...')` and rely on the gateway's `[EnableCors("BankClient")]` policy.

3. Switch ↔ Gateway (gRPC)

The synergy-switch service handles POS/ATM traffic. It speaks two wire protocols **inbound**:

- ISO 8583 over persistent TCP (legacy national networks)
- ISO 20022 over gRPC (modern smart POS terminals)

…and a single protocol **outbound** to the gateway: gRPC against `gateway:1111` using the same `CardTransactionService` contract. This lets the gateway treat all card activity uniformly regardless of which terminal type initiated it.

The switch routes **on-us** transactions (cardholder + merchant both belong to this bank — detected via BIN prefix matching) directly to the gateway, bypassing the national network. **Off-us** transactions are routed out via the configured gateway-pool to Zimswitch.

4. Notifications (in-process message bus + sidecar)

When a domain event fires inside a module (e.g. `UserAuthenticated`, `AccountCreated`, `LoanApproved`), the handler publishes it to a **Wolverine** in-process bus. A handler in `GoldBank.Notifications` picks it up, resolves a template, applies rate limits and user preferences, and dispatches via SMS gateway and/or Firebase FCM. The outbox-pattern table in PostgreSQL ensures at-least-once delivery across service restarts.

5. AI features (OpenAI-compatible client → Ollama)

KYC document OCR, asset receipt extraction, fraud-alert explanations, and chat all go through `OllamaClient` which speaks the OpenAI-compatible API to a local Ollama server (container `goldbank-ollama`) running the **qwen3-vl** vision-language model. Image inputs are base64-encoded JPEG; prompts are deterministic templates per use case.

Deployment topology

Containers are orchestrated by `docker-compose.yml` with three profiles:

- `core` — postgres, redis, gateway, switch, admin, hsm, notifications,

switch-migrator, server-migrator. This is what you bring up to develop against.

- `ai` — ollama (large image, separate profile so you can dev without it).
- `monitoring` — prometheus, grafana, elasticsearch, kibana (off by default).

The two React dev servers (`bank-client`, `bank-teller`) run **on the host** under Vite, not in containers. They proxy `/api` requests to the gateway on `localhost:5001`. See operations.md for the full port map.

Modular monolith, not microservices

Every business module lives inside one .NET assembly (`GoldBank.Core`) and shares one DbContext (`GoldBankDbContext`). The benefits:

- One transaction across modules (no distributed sagas)
- One schema, one migration set
- One container to rebuild

The costs:

- Strong coupling — changing one module's entity rebuilds the gateway image
- Single point of failure — gateway down ⇒ everything down

We've explicitly traded operational simplicity for a tighter dev loop. If any module grows enough to warrant extraction (Reporting is the likely candidate), the wire contract is already a clean gRPC service so the cut is straightforward.

Where the boundaries are

Inside the .NET solution the module folders follow a consistent layout:

```
Modules/<Name>/
  Domain/
    Entities/      ← aggregate roots + value objects
    ValueObjects/
  Application/
    Commands/      ← request DTOs
    Handlers/      ← orchestration (Wolverine handlers OR plain services)
    Validators/    ← FluentValidation
    Interfaces/    ← external dependencies (SMS gateway, OTP service)
  Infrastructure/
    Persistence/   ← EF Core IEntityTypeConfiguration<T>
    Services/      ← concrete impls (BCrypt PIN hasher, JWT generator)
  Grpc/            ← gRPC service implementation (XxxServiceBase)
```

The convention isn't religiously enforced — older modules pre-date it — but new modules follow it. The Asset Custody, Customer, and Ekub modules are the cleanest examples.

Cross-cutting concerns

Concern	Implementation
AuthN (mobile)	JWT issued by `JwtTokenService` after PIN verify; bearer header on every gRPC call; auto-refresh client-side
AuthN (web admin)	Hardcoded `SEED_ACCOUNTS` in `bank-client/src/auth/roles.js` — dev stub, not real
AuthN (teller)	JWT issued by `/api/teller/auth/login`; bearer on every REST call
AuthZ	`[Authorize(Roles = "...")]` on controllers; `ProtectedRoute` + `hasRole()` on web; role from JWT claim
Tenant isolation	`tenant_id` column on every aggregate; gRPC interceptor sets `x-tenant-id`; controllers filter
Idempotency	Outbox table for events; receipt numbers unique per (deposit_house, receipt_number); Ekub fees unique per (group, period)
Audit	`bank.audit_logs` table; admin actions on customers / loans / disputes / config writes a row

Concern	Implementation
Money	Decimal in DB and domain; string-over-wire in protos; `Money { amount, currency }` value object
Phones	`PhoneNumber` value object validates E.164 (Southern African codes)
PINs	BCrypt hash, work factor 11; never logged; `PinHashingService`
Lockout	Redis-backed counter; configurable max attempts + lockout minutes via `system_configs`

Data flow examples

A customer transfers $100 to another customer

```
phone PIN ■■■ AuthenticateHandler ■■■ JWT
       ■                              ■
mobile ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
       ■
       ■  TransferGrpcService.SendP2P (bearer = JWT)
       ▼
GatewayTransferService
       ■■■ TransactionAuthorizationService  (daily/monthly limits, fraud rules)
       ■■■ FraudRuleEvaluator               (velocity, geo, device)
       ■■■ AccountRepository.DebitCredit    (in one EF transaction)
       ■■■ IMessageBus.Publish(TransferCompleted)
                                       ■
                                       ▼
                           NotificationOrchestrator  (SMS to both parties)
                           FraudAlertEvaluator       (post-tx check)
```

A loan is approved by Ekub vote

```
mobile (borrower) ■■■ ApplyForLoan
                          ■
                          ■■■ validate pot ≥ principal + pending loans
                          ■■■ insert ekub_loans row status=Voting

mobile (members) ■■■ VoteOnLoan
                          ■
                          ■■■ insert ekub_loan_votes
                          ■■■ if approves ≥ majority of non-borrower members:
                              loan.status = AwaitingTreasurer

mobile (treasurer) ■■■ ConfirmLoanByTreasurer
                          ■
                          ■■■ re-check pot ≥ principal (defence in depth)
                          ■■■ loan.status = Disbursed
                          ■■■ ComputePotBalanceRawAsync now subtracts principal
```

A teller withdraws an asset

```
teller ■■■ withdrawAsset(assetId)
             ■
             ■■■ find every non-Closed/non-Defaulted loan
             ■   across the customer's sibling accounts
             ■
             ■■■ IF any loan's CollateralAssetIds contains assetId:
             ■      reject 409 { blockingLoan: { reference, outstanding } }
             ■
             ■■■ ELSE:
                   asset.Status = Released
                   asset.IsDeleted = true (soft-delete)
                   asset.DeletedBy = tellerId
```

Data Model

All persistent state lives in PostgreSQL on `schema = bank` (server) and `schema = synergy_switch` (switch). EF Core owns the migrations.

Multi-tenancy

Every aggregate carries a `tenant_id` column. Demo data uses the text value `"goldbank"`. Mobile registrations and teller-issued JWTs both carry this string. The gateway enforces tenant gates in two places:

- `AdminApiController` and `TellerApiController` — explicit

`if (account.TenantId != TenantId) return Forbid()` on customer/account reads.

- **gRPC services** — read `ITenantProvider.GetTenantId()` from the

`x-tenant-id` header and either filter by it or store it on writes.

Two tenant column types coexist:

Column type	Used by	Stored value
`text`	`accounts`, `customers`, `loans`, `merchants`, `bill_*`, `ekub_*`, `admin_users`, ...	`"goldbank"`
`uuid`	`assets`, `deposit_houses`, `asset_valuations`, `daily_prices`, `branches`	`00000000-0000-0000-0000-000000000000` (placeholder)

The mixed shape is historical — newer modules picked `uuid` thinking `tenant_id` would always be a Guid; older modules were already using text. There's no functional gap (the asset module fall back to `Guid.Empty` when `ParseTenantId` fails), but a future cleanup migration could re-type the uuid columns to text for consistency.

Customer aggregate (the "person not account" refactor)

A `Customer` is a person. Each customer has **one or more** `Account` **rows** — typically two, one per currency (ZWG + USD). Assets and Ekub contributions are scoped to the customer, **not** the account, so they're independent of which currency the user is "in" at the moment.

```
Customer  (1 person, identified by phone within a tenant)
   ■
   ■■ Account ZWG    (balances, card PAN, daily limits)
   ■■ Account USD    (balances, card PAN, daily limits)
   ■
   ■■ Asset 0a..0001  (Krugerrand × 2)        ■■■ owned at Customer level
   ■■ Asset 0a..0002  (Gold Eagle × 1)
   ■■ Asset 0a..0003  (Maple Leaf × 3)
   ■
   ■■ EkubMembership in Borrowdale Savings   ■■■ group savings
```

This was added in the `20260430231033_AddCustomersAggregate` migration. Before it, assets pointed at `account_id` and the customer had to pick "which account does the gold belong to?" — which made no sense for non-currency-specific assets. The migration creates customers from the existing `(tenant_id, phone)` pairs in accounts, then re-FKs everything.

Schema overview

Run `\dt bank.*` in psql for the current 50+ table list. The ones a developer needs to remember:

Identity / accounts

Table	Aggregate	Purpose
customers	Customer	Person; FK target for accounts + assets + ekub
accounts	Account	Currency-bound balance container; PIN + daily limits
refresh_tokens	RefreshToken	JWT refresh tokens, one per device
device_transfer_requests	DeviceTransferRequest	OTP-mediated "I lost my phone" flow
admin_users	AdminUser	Internal staff (admin, kyc, fraud, support, branch, teller)

Transactions / money movement

Table	Purpose
transactions	Unified ledger row for every money movement (P2P, BillPay, CashIn/Out, etc.)
transfers	P2P-specific extension data
payments	Merchant payment events
payment_tokens	Tokenised card data
card_transactions	ATM/POS card events from the switch
bill_payments	Utility bill payments
bill_providers	ZESA, TelOne, Econet, etc.
saved_billers	A user's saved utility accounts
agent_floats	Agent's cash balance for cash-in/out
agent_commissions	Agent fee accrual

KYC / compliance

Table	Purpose
kyc_documents	National ID, passport, drivers' licence, selfie, proof-of-address
kyc_verifications	AI verification result per document
disputes	Customer-raised disputes against transactions
transaction_disputes	Per-transaction extension to disputes
fraud_alerts	Suspicious activity flagged by rules engine
fraud_rules	Rule definitions (velocity, geo, device, pattern, amount)
audit_logs	Internal-staff action log

Assets / gold custody

Table	Aggregate	Purpose
customers (FK)	—	Owner of the asset
deposit_houses	DepositHouse	Trusted vault facility (license, contact, trust status)
assets	Asset	A specific physical asset in custody (qty, weight, purity, receipt #)
asset_valuations	AssetValuation	History of professional valuations
daily_prices	DailyPrice	Spot price per gram for gold/silver/platinum

Loans

Table	Purpose
`loans`	`Loan` aggregate root (principal, rate, tenure, status, `collateral_asset_ids` JSON list)
`loan_payments`	Repayment installments

Ekub (group savings + lending)

Table	Purpose
`ekub_groups`	Group (name, currency, monthly amount, loan rate, `apply_interest_on_contributions`)
`ekub_memberships`	Customer + role (Chairman/Treasurer/Secretary/Member)
`ekub_invitations`	Pending or resolved invitations (status, expiry)
`ekub_contributions`	A member's contribution; treasurer confirms
`ekub_fees`	Monthly bank fee debit per (group, period)
`ekub_loans`	A member's loan against the pot
`ekub_loan_votes`	One row per voter per loan (borrower excluded)
`ekub_loan_repayments`	Split into principal + interest portions

Branch / vault operations

Table	Purpose
`branches`	Physical branch (name, code, city)
`teller_drawer_sessions`	Open/close shifts; cash counts
`branch_cash_transactions`	Per-shift cash movements
`currency_denominations`	Configurable per-tenant denomination list
`vaults`	Branch vault (1 per branch)
`vault_denomination_stock`	Notes/coins on hand per vault
`vault_movements`	Cash in/out of vault (with denominations breakdown)
`vault_spot_checks`	Periodic count vs ledger reconciliation

Merchants / acquiring

Table	Purpose
`merchants`	POS-accepting business
`merchant_documents`	Onboarding docs
`merchant_settlements`	Daily settlement batches

Configuration

Table	Purpose
`system_configs`	Key/value JSON config (PIN policy, fraud thresholds, fee config, etc.)
`tenant_branding`	Per-tenant logo, colours, name
`tenant_fee_configs`	Per-tenant transaction fees
`tenant_transaction_limits`	Per-tenant daily/monthly caps

Async / events

Table	Purpose
`outbox_messages`	Wolverine outbox pattern — events to dispatch
`cache_entries`	Application-side cache (in addition to Redis)

Key invariants

These are encoded as unique indexes or check constraints. Worth keeping in your head:

- **One customer per phone within a tenant**: `ix_customers_tenant_phone_unique`.
- **One account per phone-currency within a tenant**: `ix_accounts_phone_currency_unique`.
- **One asset per (deposit_house, receipt_number)**: `ix_assets_deposit_house_receipt_unique`.

Stops the same physical receipt being recorded twice.

- **One Ekub membership per (group, customer) when active**:

`ix_ekub_memberships_group_customer_active_unique` (filtered on `left_at IS NULL`).

- **One Ekub fee per (group, period)**: `ix_ekub_fees_group_period_unique`.

Makes the monthly-fee job idempotent.

- **One vote per (loan, voter)**: `ix_ekub_loan_votes_loan_voter_unique`.

Members can change their vote (UPDATE), not stack votes.

- **One pending invitation per (group, phone)**: filtered partial unique

index on `status = 'Pending'`.

- **Soft-delete via** `is_deleted` **+** `deleted_at` on `assets`; via

`deleted_at` IS NOT NULL on accounts/customers/loans. Active queries always include the not-deleted filter.

Outbox / event flow

Domain events declared in `SharedKernel/Events/` are published through `IMessageBus` (Wolverine). The outbox table guarantees delivery across restarts:

```
1. EF transaction commits
   ■■ business state change          (accounts.balance -= 100)
   ■■ row inserted into outbox       (event payload + status='pending')
2. OutboxProcessor (background svc)
   ■■ polls every 5 s
   ■■ dispatches to in-process handlers
   ■■ marks row processed
```

`OutboxProcessor` is started in `Program.cs` at gateway boot. The notifications service reads from the same bus.

Migrations

Located at `server/GoldBank.Migrator/Migrations/UniBankDb/`. Folder name still uses the old "UniBank" prefix (rename pending; namespace inside is `GoldBank.Migrator.Migrations.UniBankDb`). Each migration is a hand-edited or `dotnet ef migrations add`-generated pair (.cs + .Designer.cs) plus a single shared `GoldBankDbContextModelSnapshot.cs` snapshot file.

Recent migrations of note:

Migration	What it does
20260430231033_AddCustomersAggregate	Creates bank.customers, backfills from accounts, re-FKs assets from account_id → customer_id
20260501063756_AddEkubModule	5 tables for Ekub v1 (groups, memberships, invitations, contributions, fees)
20260501070815_AddEkubLoans	3 tables for Ekub v2 (loans, votes, repayments)
20260501153434_AddEkubInterestOptOut	Adds apply_interest_on_contributions to ekub_groups
20260430120000_AddLoanCollateralAssetIds	Adds JSON column to loans for asset-backed lending

Apply with `dotnet run --project server/GoldBank.Migrator -- --context GoldBankDb`.

Demo data

`server/GoldBank.Migrator/DemoSeeder.cs` is idempotent and runs when `--demo` is passed:

```
 20 customers + 40 accounts        (Tendai Moyo, Chiedza Mutasa, …)
  8 admin users                    (admin / kyc / fraud / support / loans / compliance / branch / teller; password = use
  5 loans                          (pending; mix of customers)
  8 merchants                      (OK Supermarket, TM Pick n Pay, etc.)
  5 bill providers                 (ZESA, TelOne, NetOne, Econet, ZINWA)
 50 transactions, 15 disputes, 12 fraud alerts, 6 KYC docs, 60 audit logs
  6 branches                       (HQ, Borrowdale, Bulawayo Main, Mutare, Gweru, Vic Falls)
  1 Ekub group (Borrowdale Savings) + 5 memberships + 3 contributions + 1 voting loan
 22 system_configs                 (limits, fraud thresholds, OTP TTL, Ekub monthly fees, etc.)
```

Additional one-off SQL seeds in `scripts/`:

- `seed-gold-coins.sql` — 1 deposit house + 5 gold-coin assets for John Moyo
- `seed-asset-valuations.sql` — 10 valuation rows (initial + revalued) so the

history tab shows % change

- `age-asset-valuations.sql` — backdates 2 valuations so the Valuation Queue

has overdue items

Connection string

Local dev: `Host=localhost;Port=5432;Database=goldbank;Username=goldbank;Password=goldbank_dev_password`

Container-to-container: `Host=postgres;Port=5432;...` (same credentials). Both the gateway and the switch share the same Postgres instance but different databases (`goldbank` and `synergy_switch` respectively).

Server

The server is a **modular monolith** in .NET 10 — one `GoldBank.Gateway` executable hosts gRPC + REST endpoints; one `GoldBank.Core` library houses every business module; one `GoldBankDbContext` owns the schema; one PostgreSQL database backs the lot.

Solution structure:

```
server/
  GoldBank.SharedKernel/    base classes, value objects, message bus, results
  GoldBank.Core/            domain logic (Modules/<Name>/...)
  GoldBank.Protos/          *.proto definitions → C# stubs at build
  GoldBank.Gateway/         hosting + Kestrel + controllers + Program.cs
  GoldBank.Notifications/   SMS / FCM sidecar
  GoldBank.Reporting/       reports (separate assembly so it can be split off)
  GoldBank.Migrator/        EF migrations + DemoSeeder + CLI utilities
```

Endpoints at a glance

The gateway exposes two wire surfaces. Both are served by the same Kestrel host, listening on two different ports.

gRPC (port 1111 inside container, host :5000)

For mobile and switch traffic. Registered in `Program.cs` via `app.MapGrpcService<T>()`:

Service	File	Used by
`AccountService`	`Modules/Accounts/Grpc/AccountGrpcService.cs`	mobile (register, OTP, PIN, login, refresh, profile, balance, transactions, device-transfer)
`PaymentService`	`Modules/Payments/Grpc/PaymentGrpcService.cs`	mobile (QR generate/scan, NFC payment)
`TransferService`	`Modules/Transfers/Grpc/TransferGrpcService.cs`	mobile (P2P transfers, history)
`BillPayService`	`Modules/BillPay/Grpc/BillPayGrpcService.cs`	mobile (provider list, pay bill, saved billers)
`AgentService`	`Modules/Agents/Grpc/AgentGrpcService.cs`	mobile (cash-in / cash-out at agents)
`KycService`	`Modules/KYC/Grpc/KycGrpcService.cs`	mobile (upload ID, selfie, POA)
`MerchantService`	`Modules/Merchants/Grpc/MerchantGrpcService.cs`	mobile (merchant onboarding + dashboard)
`WhiteLabelService`	`Modules/WhiteLabel/Grpc/WhiteLabelGrpcService.cs`	mobile (tenant branding)
`LoanService`	`Modules/Loans/Grpc/LoanGrpcService.cs`	mobile (apply, list, detail, repay)
`CardTransactionService`	`Modules/CardTransactions/Grpc/CardTransactionGrpcService.cs`	**switch** (POS/ATM authorisations)
`AdminService`	`Modules/Admin/Grpc/AdminGrpcService.cs`	(legacy — admin web used to use this; bank-client now uses REST)
`AiService`	`Modules/AI/Grpc/AiGrpcService.cs`	mobile (chat, OCR, insights)
`ReportingService`	`Reporting/Grpc/ReportingGrpcService.cs`	bank-client reports
`AssetService`	`Modules/AssetCustody/Grpc/AssetGrpcService.cs`	mobile (register/list/release assets, valuation history, daily prices)
`EkubService`	`Modules/Ekub/Grpc/EkubGrpcService.cs`	mobile (groups, contributions, loans, voting)

Each `.proto` lives at `server/GoldBank.Protos/Protos/<service>_service.proto`. The `Grpc.Tools` package regenerates C# stubs on every build.

REST (port 1112 inside container, host :5001)

For the React web apps. Controllers under `GoldBank.Gateway/Controllers/`:

Controller	Prefix	Purpose
`AdminApiController`	`/api/admin`	bank-client read endpoints + admin actions
`TellerApiController`	`/api/teller`	bank-teller endpoints (auth-gated)
`TellerAuthController`	`/api/teller/auth`	teller JWT login

REST is read-heavy by design. State changes still mostly happen through gRPC; REST endpoints either project from the DB (`AdminApi.GetCustomers`) or wrap an existing module command (`TellerApi.PostAssetValuation` mirrors `AssetGrpcService.SubmitValuation`).

Modules

Each module owns one or more **aggregate roots** plus the gRPC service (if exposed). Listed alphabetically below; the order is not architectural.

Accounts

`Modules/Accounts/` — authentication, account lifecycle, transactions.

Aggregates: `Account`, `Transaction`, `RefreshToken`, `DeviceTransferRequest`.

Handlers worth knowing:

- `RegisterHandler` — phone-validation, OTP issue. Returns `registrationId`.
- `VerifyOtpHandler` — validates OTP, creates **Customer + dual-currency

accounts** atomically, issues a temporary token.

- `CreatePINHandler` — sets BCrypt PIN hash on all of a customer's accounts

(so PIN is one-per-person, not one-per-account), generates a card PAN, issues the real JWT pair.

- `AuthenticateHandler` — phone+PIN login. Lockout via `LockoutService` (Redis).
- `RefreshTokenHandler` — exchange refresh token for a new access token.
- `DeviceTransferHandler` — "I have a new phone" — OTP-protected device rebind.

Cross-module: publishes `UserRegistered`, `AccountCreated`, `PINCreated`, `UserAuthenticated`. Notifications subscribes.

Customers

`Modules/Customers/` — the person aggregate. One row per `(tenant_id, phone)`.

Tiny module by design — `Customer.cs` is 14 properties. Doesn't expose gRPC of its own; created by `VerifyOtpHandler` and updated by `UpdateProfileHandler`.

Asset Custody

`Modules/AssetCustody/` — physical asset (gold, silver, platinum, gemstones) custody, valuation, and release.

Aggregates: `Asset`, `DepositHouse`, `AssetValuation`, `DailyPrice`.

Key gRPC RPCs:

- `RegisterAsset` — customer registers an asset deposit (with a receipt #).
- `ListMyAssets`, `GetAssetDetail` — read flow.
- `RequestAssetRelease` — customer asks to withdraw. Marks `PendingRelease`.
- `SubmitValuation` (admin) — record a valuation; if asset was

`PendingVerification`, promote to `Active`.

- `VerifyCertificate` (admin) — record a certificate-of-authenticity check.
- `ApproveAssetRelease` (admin) — physically release; soft-deletes the row.
- `GetPortfolioValue` — sum across all assets, in ZWG + USD.

Two REST counterparts in `TellerApiController` for the branch flow:

- `POST /api/teller/customers/{accountId}/assets` — teller-mediated deposit.
- `POST /api/teller/assets/{assetId}/withdraw` — release at the counter,

blocked when the asset is **active collateral** on a loan (see Loans).

- `POST /api/admin/asset-valuations` — admin/valuer submits a valuation.

Pricing logic in `AssetValuationService`: `quantity × weight_grams × purity × spot_price_per_gram` for metals; falls back to the most recent recorded valuation amount for `PreciousStone` / `Other`.

Loans

`Modules/Loans/` — straight-bank loan origination (not Ekub loans, which live in their own module).

Aggregate: `Loan`, `LoanPayment`.

`Loan.CollateralAssetIds` is a `List<Guid>` serialised to a JSON `text` column. When a loan is taken with collateral, the asset IDs land here. `TellerApiController.WithdrawAsset` cross-references this when releasing assets — see bank-teller.md.

Loans use a `string` status field (not an enum) for historical reasons: `pending` → `approved` → `disbursed` → `repaying` → `closed` / `defaulted` / `rejected`. Cleanup to a proper enum is pending.

Ekub

`Modules/Ekub/` — group savings + lending. See `EkubGroup`, `EkubMembership`, `EkubInvitation`, `EkubContribution`, `EkubFee`, `EkubLoan`, `EkubLoanVote`, `EkubLoanRepayment`.

The full RPC surface (15 RPCs):

```
Group lifecycle:     CreateGroup, AssignRole, CloseGroup
Membership:          InviteMember, RevokeInvitation, ListMyInvitations,
                     RespondToInvitation, KickMember
Contributions:       RecordContribution, ConfirmContribution,
                     ListGroupContributions
Reads:               ListMyGroups, GetGroupDetail, GetMyShare
Monthly fee:         ApplyMonthlyFee
Loans (v2):          ApplyForLoan, VoteOnLoan, ConfirmLoanByTreasurer,
                     RecordLoanRepayment, ListGroupLoans, ListMyLoans,
                     GetLoanDetail
```

Key rules baked into the service:

- **Quorum to activate**: a group needs ≥ **3 active members** before it

flips from `Forming` to `Active`. Auto-promoted on the 3rd acceptance.

- **Role assignment**: only `Chairman` can call `AssignRole`. Chairman /

Treasurer / Secretary roles are **unique** per group (assigning to a new member demotes the prior holder to plain Member).

- **Invitations**: only `Chairman` or `Secretary` can invite. 30-day TTL.

No duplicate pending invites per (group, phone).

- **Contributions**: any active member submits. Only `Treasurer` can

confirm. Pending contributions don't count in pot until confirmed.

- **Loan eligibility**:
- Group must be `Active`.
- Borrower must be an active member.
- One open loan per borrower per group.
- Pot ≥ principal **net of pending loans** (Voting + AwaitingTreasurer).
- **Voting**:
- Borrower **cannot** vote on their own loan.
- Strict majority of non-borrower active members approves

(`floor(eligible/2) + 1`). Treasurer's vote counts as a normal member vote — the separate "treasurer confirmation" step is what disburses.

- **Treasurer disbursement**: at confirm time the pot is **re-checked**;

if it can't cover the principal (e.g. fees were applied or another loan disbursed since application), the confirmation fails.

- **Interest math** (`ApplyForLoan`):

`` `if group.ApplyInterestOnContributions: interestable = principal else: interestable = max(0, principal − borrower_confirmed_contributions) totalInterest = interestable × rate% / 100 × term_months / 12 totalRepayable = principal + totalInterest installment = totalRepayable / term_months` `` So with the flag off, a member borrowing within their own contributions pays zero interest.

- **Repayment split** (`RecordLoanRepayment`): only `Treasurer` may

record. Each payment is split into principal vs interest by the loan's original ratio. Interest goes into `loan.TotalInterestEarned` and redistributes pro-rata across all members' shares via `GetMyShare`.

- **Pot balance**:

`` `pot = sum(confirmed_contributions) − sum(fees) − sum(disbursed loan principals) // Disbursed/Repaying/Closed/Defaulted + sum(repayments_amount_paid)` Defined in ComputePotBalanceRawAsync`.

- **My share** (per-member):

`` `my_contributions = sum(my confirmed contributions) my_interest = total_repayment_interest × (my_contributions / total_confirmed) my_share_total = my_contributions + my_interest` ``

KYC

`Modules/KYC/` — `KycDocument` + `KycVerification`.

Files are stored **inline in the DB** (`file_data BYTEA`) for the demo; in prod this would be S3/MinIO. Verification is async — a Wolverine handler calls `OllamaClient.ExtractFromImageAsync<IdentityFields>(...)` and records confidence per field. `kyc.face_match_auto_approve` (0.80 by default) and `kyc.face_match_reject` (0.40) thresholds gate auto-decisions.

BillPay

`Modules/BillPay/` — `BillProvider`, `SavedBiller`, `BillPayment`.

The bill providers are seeded (ZESA, TelOne, NetOne, Econet, ZINWA). A real provider integration would inject `IProviderClient` per provider; the demo writes a transaction row and acks.

Fraud Detection

`Modules/FraudDetection/` — `FraudAlert`, `FraudRule`.

Five rule types, all evaluated in-line by `FraudRuleEvaluator` during transaction authorization:

- **Velocity**: > N transactions in M minutes.
- **GeoAnomaly**: device location differs from prior pattern.
- **DeviceAnomaly**: new device with a high-value transaction.
- **AmountAnomaly**: amount exceeds the per-user statistical threshold.
- **PatternMatch**: signature against known mule/scam patterns.

Thresholds live in `system_configs` under `fraud.*` keys. Alerts have a status field (`New` → `Reviewed` → `Escalated` / `Dismissed`). Fraud analysts triage from the bank-client UI.

Branch Cash & Vault

`Modules/BranchCash/` — physical cash management at branches.

- `TellerDrawerSession` — a teller's shift, opened + closed with cash

counts. EOD report PDF generated at close.

- `BranchCashTransaction` — every cash in/out at the counter, with a

denomination breakdown.

- `Vault`, `VaultDenominationStock`, `VaultMovement`, `VaultSpotCheck` —

the branch vault layer above tellers.

- `CurrencyDenomination` — configurable per-tenant denomination list (note

and coin face values).

The teller / vault / supervisor dashboards in `bank-teller/` consume this module's REST endpoints under `/api/teller/`.

Card Transactions

`Modules/CardTransactions/` — receives card auth requests from the **switch** and posts to the appropriate account. ATM withdrawals, POS purchases, refunds. The on-us vs off-us routing decision happens in the switch, not here.

Admin

`Modules/Admin/` — internal staff users, audit logs, system configs, disputes.

`AdminUser` includes the staff JWT identity (BCrypt password hash, role, branch ID for tellers/branch-managers). `audit_logs` is append-only. `system_configs` is the key/value JSON store for runtime tuneables.

White Label

`Modules/WhiteLabel/` — `TenantBranding`, `TenantFeeConfig`, `TenantTransactionLimit`. The platform is multi-tenant by design (one gateway, N tenants); each tenant gets a logo + colours + per-currency limits.

Reporting

`server/GoldBank.Reporting/` — six report types, one streaming export. Lives in its own assembly so it can be lifted into a separate process later. Exposed as `ReportingService` gRPC; surfaced in the bank-client "Reports" pages.

RPC	Returns
`GetDashboard`	Live counters (users, txns, volume, merchants, agents, loans)
`GetUserGrowth`	Registrations + churn by day/week/month
`GetMerchantReport`	Volume + commission per merchant
`GetRevenueReport`	Revenue breakdown by transaction type
`GetReconciliationReport`	Settlement vs ledger discrepancies
`Export`	Streams CSV chunks for the active report

AI

`Modules/AI/` — single `OllamaClient` that speaks OpenAI-compatible API. Use cases:

Handler	Prompt purpose
`VerifyIdentityHandler`	Extract ID fields from a national ID / passport image
`VerifyProofOfAddressHandler`	Validate name + address on a utility bill
`ExtractReceiptHandler` (asset custody)	Parse a safe-deposit receipt photo
`ExtractBillFieldsHandler`	Parse a utility bill for amount + meter #
`ExtractChequeFieldsHandler`	Parse cheque amount + payee
`ExtractDocumentFieldsHandler`	Generic document OCR
`ChatHandler`	In-app assistant (RAG-style, conversation history)
`GetSpendingInsightsHandler`	Generate spending summary from transaction history
`ExplainFraudAlertHandler`	Human-readable explanation of why an alert triggered
`TriageDisputeHandler`	Auto-classify dispute type
`CheckLoanEligibilityHandler`	Pre-screen a loan application
`VerifyLoanDocumentsHandler`	Validate uploaded supporting docs

The `ai.ollama_url` + `ai.model_name` system_configs control the endpoint and model (default: `qwen3-vl`). AI calls are best-effort with a circuit breaker — if Ollama is unreachable, the calling handler falls back to a "pending manual review" state.

Conventions when adding to the server

1 **New aggregate** → new folder under `Modules/<Name>/Domain/Entities/`

+ an `IEntityTypeConfiguration<T>` under `Infrastructure/Persistence/`. Add `modelBuilder.ApplyConfiguration(new TConfig())` in `GoldBankDbContext.OnModelCreating`.

1 **Run** `dotnet ef migrations add <Name> --context GoldBankDbContext

--output-dir Migrations/UniBankDb `from` server/GoldBank.Migrator/`.

1 **New gRPC service** → add a `.proto` under `GoldBank.Protos/Protos/`.

Rebuild — stubs regenerate. Implement `XxxServiceBase`. Map in `GoldBank.Gateway/Program.cs`: `app.MapGrpcService<XxxGrpcService>();`.

1 **New REST endpoint** → add to `AdminApiController` (bank-client) or

`TellerApiController` (bank-teller). Use `[Authorize(Roles = "...")]`.

1 **Mobile-facing changes** → also update `mobile/shared/.../grpc/...` Kotlin

client + `mobile/shared/.../mapper/...` + any view model.

1 **Money** stays as `decimal` in the domain, `string` on the wire.

2 **Commit + rebuild gateway image** for the change to land in the

running container: `podman build -f server/GoldBank.Gateway/Dockerfile -t localhost/goldbank_gateway:latest .`

Common gotchas

- **EF can't translate JSON-column** `Contains` for `List<Guid>` columns

like `loans.CollateralAssetIds`. Pull the rows into memory and filter in C# (`AsEnumerable().Where(...)`).

- **Tenant gates use string equality**. If you stage data with a UUID

tenant_id (`"00000000-..."`) the teller's `tenant = "goldbank"` JWT won't see it — 403. See data-model.md.

- `MapLoanResponseAsync` **needs the requesterId** to populate `my_vote`.

Pass it from every call site that knows who's asking (5 of the 7).

- **Notifications fail silently** if Wolverine can't reach Redis. Check

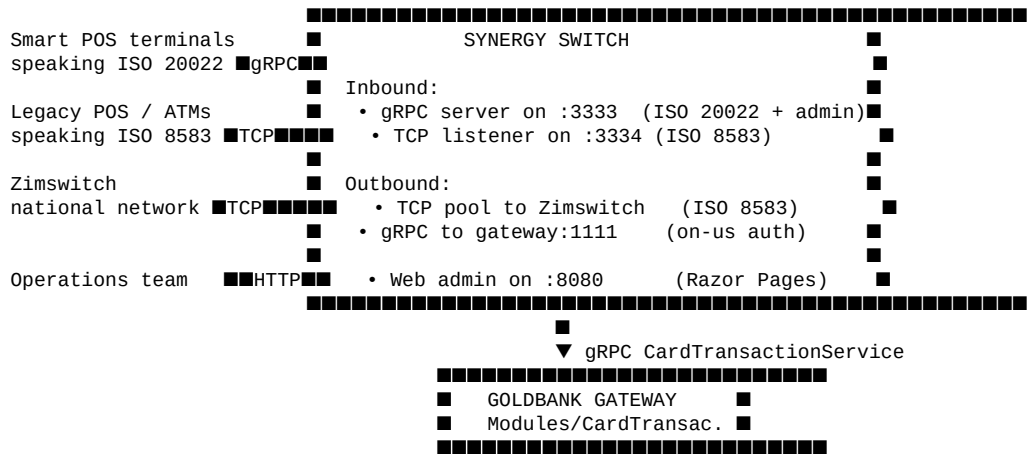
the gateway logs for `OutboxProcessor` errors when "the SMS didn't come through".

- **AI calls are slow** (Qwen3-VL is 8B params). Don't block UI on them;

use the "pending review" pattern in the calling handler.

SynergySwitch

`switch/` is a satellite service that bridges card-acceptance hardware (POS terminals, ATMs) and national payment networks (Zimswitch) to the GoldBank gateway. It runs as a separate container (`goldbank-switch`) but shares the same Postgres instance (different DB: `synergy_switch`).



Why it exists

A core-banking gateway shouldn't know about ISO 8583 wire frames or terminal-vendor quirks. The switch:

1. **Normalises wire protocols.** ISO 8583 (TCP, MTI + bitmap, BCD-encoded

PAN) and ISO 20022 (JSON over gRPC) both become a single `CardTransactionAuthorisation` request on the gateway side.

1. **Routes on-us vs off-us.** A transaction where both cardholder *and*

merchant are GoldBank customers stays local (gateway gRPC). One where the cardholder is on a foreign bank (Stanbic, Steward, NMB, …) gets shipped out over ISO 8583 to Zimswitch.

1. **Connection pooling.** ISO 8583 partner links are persistent TCP

sockets. The switch holds the pool — the gateway never sees it.

1. **Settlement.** End-of-day batches are reconciled against Zimswitch

files inside the switch's own DB; the gateway only consumes the summary.

On-us / off-us routing

`Routing/BinResolver.cs` does longest-prefix BIN matching:

```
Configured BIN ranges:
  6275 00 ……  → on-us   (GoldBank issuer ID)
  6275 12 ……  → off-us  (Stanbic — same first 4 but different range)
  4000 00 ……  → off-us  (Visa)
  5100 00 ……  → off-us  (Mastercard)

A transaction's PAN:
  6275 0012 3456 7890   ← on-us  (matches 6275 00 longer than any other)
  6275 1234 5678 9012   ← off-us
  4000 1111 2222 3333   ← off-us
```

The card BIN prefix list is seeded in `system_configs` under `card.bin_prefix` (a JSON-stringified `"6275"` default) and extended at runtime by ops staff via the switch's web admin. On-us transactions short-circuit national-network egress, which has two benefits:

- **Lower interchange.** Zimswitch charges per-transaction; on-us avoids

it.

- **Faster auth.** Skip a TCP round-trip + network ISO parse.

Ports

Port	Protocol	Direction	Used by
3333	gRPC (h2c)	inbound	smart POS terminals, gateway admin operations
3334	TCP (ISO 8583)	inbound	legacy POS / ATMs / national link
8080	HTTP (Razor Pages)	inbound	switch operations dashboard
1111	gRPC (h2c)	**outbound**	calls the gateway's `CardTransactionService`

In `docker-compose.yml` these map to host ports as `SWITCH_GRPC_PORT=3333`, `SWITCH_WEB_PORT=8080`. The ISO 8583 port isn't exposed to the host by default — it's intra-container only.

Database

The switch owns the `synergy_switch` PostgreSQL database (separate from the bank's `goldbank` DB; same instance). Tables of note:

- `terminals` — registered POS / ATM hardware (terminal ID, merchant,

certificate fingerprint).

- `bin_ranges` — runtime-editable list of card BIN prefixes and their

destinations.

- `transactions` — the switch's own transaction log; cross-references the

gateway's `card_transactions` table via `external_id`.

- `iso8583_messages` — raw wire frames (for forensic / dispute use).
- `settlement_batches` — daily files received from Zimswitch.

Migrations are applied by the `goldbank-switch-migrator` container that runs once at compose-up time.

Configuration

Env var	Default	Purpose
`BankConnection__Host`	`gateway`	gRPC target for the gateway
`BankConnection__Port`	`1111`	gRPC port for the gateway
`BankConnection__OfflineMode`	`false`	Buffer authorisations locally when the gateway is down
`Kestrel__Endpoints__Grpc__Url`	`http://0.0.0.0:3333`	Inbound gRPC bind
`Kestrel__Endpoints__Web__Url`	`http://0.0.0.0:8080`	Admin web bind
`ConnectionStrings__SwitchDb`	`Host=postgres;...;Database=synergy_switch;...`	Postgres

`OfflineMode` is the EFT-grade resilience knob: when `true`, the switch will accept transactions up to a configurable floor limit, cache them locally, and replay them when the gateway is back. Off by default for dev.

Talking to the switch as a developer

You generally don't. The two ways you might:

1 **Simulate a card auth** — `scripts/iso8583-simulator.py` (if present)

or any external ISO 8583 tool pointed at `localhost:3334`.

1 **Operations web** — open `http://localhost:8080` to see live

transaction log, gateway health, BIN range editor, settlement files.

The mobile / bank-client apps don't talk to the switch — they only ever talk to the gateway. The switch is purely a card-network plane.

Status

The on-disk `switch/` folder has been seen empty in some checkouts — the service is described in `docker-compose.yml`, in `docs/epics/EPIC-016-synergy-switch-integration.md`, and built via `switch/Containerfile` (the build context expects a full project tree that's not always synced with the repo HEAD). If you find an empty `switch/`, that's the EPIC-016 implementation pending merge in a topic branch.

Failure modes

Symptom	Likely cause
Gateway returns `Unavailable` from `CardTransactionService`	Switch container is up but gateway is down. Check `podman ps`.
Switch logs `ResolveBin: no match`	New BIN range needs to be added in switch admin.
ISO 8583 connections timing out	Zimswitch keep-alive failing. Restart the switch container to reset the TCP pool.
`on-us` transactions still go off-us	`card.bin_prefix` in `system_configs` doesn't match the issued card PANs. Check `account.card_pan` first digits.
Settlement files not appearing	The `goldbank-switch` container lacks volume mount for `/settlement/`. See `docker-compose.yml`.

Mobile

`mobile/` is a **Kotlin Multiplatform** app. Only the Android variant ships; the `commonMain` source set is structured for an iOS target that hasn't been built out.

```
mobile/
  shared/
    src/
      commonMain/kotlin/     pure-Kotlin domain models (no Android types)
      androidMain/kotlin/    Android-specific: gRPC clients, EncryptedSharedPreferences,
                             AuthRepositoryImpl, mappers, DI module
  androidApp/
    src/main/kotlin/com/goldbank/app/
      MainActivity.kt        single Activity host
      UniBankApplication.kt  Application class, Koin bootstrap
      navigation/NavGraph.kt route table (Type-safe routes via kotlinx.serialization)
      ui/<feature>/          Compose screens, one folder per feature area
      viewmodel/<Feature>VM  ViewModels (one per screen group)
      di/PresentationModule  Koin viewModel bindings
```

Stack

Layer	Library
UI	Jetpack Compose + Material 3
Navigation	androidx.navigation.compose (type-safe routes)
DI	Koin
Async	Kotlin coroutines
Networking	gRPC-Kotlin (`io.grpc:grpc-kotlin-stub`) + OkHttp channel
Persistence	DataStore (preferences) + EncryptedSharedPreferences (secrets)
Image / NFC	Compose Coil + AndroidX HCE
Biometrics	AndroidX Biometric

Auth flow

```
┌─ REGISTER ──────────────────────────────────────────────────────────┐
│  RegisterScreen      → AccountGrpcClient.register(phone, deviceId) │
│                    ┌─ RegistrationResult { registrationId, ttl } │
│  OtpScreen           → AccountGrpcClient.verifyOtp(otp)            │
│                    ┌─ OtpVerificationResult { accountId, tmpTok }│
```

```
■  CreatePinScreen     → AccountGrpcClient.createPin(pin, confirm)   ■
■                    ■■■ AuthTokens { access, refresh, customerId } ■
■  ProfileInfoScreen   → accountClient.updateProfile(...)            ■
■  RegistrationIdUpl   → KycGrpcClient.uploadDocument(NationalId)    ■
■  RegistrationSelfie  → KycGrpcClient.uploadSelfie(...)             ■
■  → sessionManager.logout()  (saves phone for next launch)          ■
■  → SessionState.PinRequired → AuthNavHost(startAtLogin = true)     ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■■■ LOGIN (returning) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■  LoginScreen (PIN-only; phone pre-filled, Change link to swap)     ■
■                     → AccountGrpcClient.authenticate(phone, pin)  ■
■                    ■■■ AuthResult.Success(AuthTokens)             ■
■                      ■  also: LockedOut(seconds), Failed(remaining)■
■  → sessionManager.saveTokens(tokens)                               ■
■  → SessionState.Authenticated → MainNavHost                         ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Token auto-refresh

`shared/.../data/remote/TokenRefresher.kt` is the centrepiece. Every `grpcCall { … }` invocation routes through Koin's `GlobalContext` to fetch `TokenRefresher` and call `ensureFresh()` before the actual gRPC stub call.

```
suspend fun ensureFresh() {
    if (currentCoroutineContext()[RefreshGuard] != null) return  // re-entry guard
    val refresh = sessionManager.getRefreshToken() ?: return
    if (!sessionManager.isTokenExpiringSoon()) return            // 60-s threshold
    mutex.withLock {
        if (!sessionManager.isTokenExpiringSoon()) return@withLock   // re-check
        withContext(RefreshGuard()) {
            val r = authRepository.refreshToken(refresh, deviceIdProvider())
            if (r is Result.Failure) sessionManager.logout()      // refresh denied → re-login
        }
    }
}
```

The `RefreshGuard` `CoroutineContext.Element` is the key — when the refresh call goes back through `grpcCall` (it does — `AccountGrpcClient.refreshToken` uses the same helper), the inner call sees the guard and skips the re-entry, avoiding mutex deadlock.

Visible to the user: nothing. Visible to `adb logcat -s TokenRefresher`:

```
D/TokenRefresher: Connection about to expire - refreshing
D/TokenRefresher: Connection refreshed
```

Navigation graph

Defined in `androidApp/.../navigation/NavGraph.kt`. The top-level routes are listed in `Routes.kt` as `@Serializable data object` (no args) or `@Serializable data class` (with args). The route tree:

```
AuthGraph (when SessionState in { Unauthenticated, PinRequired })
  ■■■ Register
  ■■■ Otp(registrationId, otpLength, ttlSeconds)
  ■■■ CreatePin(accountId)
  ■■■ ProfileInfo
  ■■■ RegistrationIdUpload(accountId)
  ■■■ RegistrationSelfie
  ■■■ Login(showPhoneField, initialPhoneNumber)

MainGraph (when SessionState.Authenticated)
  ■■■ Home
  ■   ■■■ TransactionList
  ■   ■   ■■■ TransactionDetail(txnId)
  ■   ■■■ Notifications
  ■   ■■■ Profile
  ■   ■   ■■■ EditProfile
  ■   ■   ■■■ SecuritySettings
  ■   ■   ■■■ NotificationSettings
  ■   ■   ■■■ DeviceTransfer
  ■   ■   ■■■ Settings
```

```
■   ■■■ (quick-action grid → next sub-routes)
■
■■■ Payments
■   ■■■ P2PTransfer
■   ■■■ QrGenerate
■   ■■■ QrScan
■   ■■■ NfcPayment
■■■ BillPay
■   ■■■ ProviderList
■   ■■■ PayBill(providerId)
■■■ CashFlow
■   ■■■ CashIn
■   ■■■ CashOut
■■■ Loans
■   ■■■ LoanList
■   ■■■ LoanApply
■   ■■■ LoanDetail(loanId)
■■■ KYC
■   ■■■ KycDashboard
■   ■■■ DocumentUpload(type)
■   ■■■ ProofOfAddress
■   ■■■ Selfie
■   ■■■ KycVerificationResult
■■■ Disputes
■   ■■■ DisputeList
■   ■■■ DisputeDetail(disputeId)
■   ■■■ DisputeWizard(transactionId)
■■■ FraudAlerts
■   ■■■ FraudAlertList
■   ■■■ FraudAlertDetail(alertId)
■■■ Merchant (for merchant accounts)
■   ■■■ MerchantRegister
■   ■■■ MerchantDashboard
■   ■■■ MerchantTransactions
■   ■■■ MerchantSettlements
■   ■■■ MerchantCommission
■■■ Scans
■   ■■■ BillScan
■   ■■■ ChequeScan
■   ■■■ ReceiptScan
■■■ Assets  (Asset Custody)
■   ■■■ AssetList
■   ■■■ AssetDetail(assetId)
■   ■■■ AssetRegister
■■■ Ekub
    ■■■ EkubGroupList
    ■■■ EkubCreateGroup
    ■■■ EkubInvitations
    ■■■ EkubGroupDetail(groupId)

ChatFAB overlay (any authenticated screen)
SessionLockScreen overlay (when inactivity timeout fires)
```

Two route keys land on `Home`'s quick-action grid that aren't directly covered above: `"assets"` → `Route.AssetList`, `"ekub"` → `Route.EkubGroupList`. The grid is in `ui/components/QuickActionGrid.kt`.

Feature deep-dives

Asset Custody (mobile side)

- `AssetListScreen` → `AssetGrpcClient.listMyAssets(customerId)` →

rendered as cards with status chips.

- `AssetDetailScreen` → `getAssetDetail` + `getDailyPrices` for the

current spot reference; shows valuation history + a "Request release" button.

- `AssetRegisterScreen` → multi-step (photograph receipt → AI OCR

extracts fields → user confirms → submit). The OCR call uses `AiGrpcClient.extractDepositReceipt` with the receipt JPEG bytes.

- `PortfolioValue` — totals in ZWG + USD; surfaced on the Home screen as

a "Custody" tile (when assets > 0).

All asset calls are scoped to `customerId` (from `SessionManager.getCustomerId()`), not `accountId`. This makes the assets visible regardless of which currency account is "active".

Ekub (mobile side)

The full UI surface:

Screen	What it does
`EkubGroupListScreen`	Lists groups the user is a member of; pending invitations badge; "Create group" CTA
`EkubInvitationsScreen`	Accept / decline pending invites
`CreateEkubGroupScreen`	Form for name, currency, monthly amount, interest rate, "charge interest on contributions" toggle
`EkubGroupDetailScreen`	Three tabs: Members / Contributions / Loans. Role-aware action buttons

On the **Group Detail** screen the role determines what's possible:

Role	Can	Cannot
Chairman	Invite, AssignRole, KickMember, CloseGroup	Confirm contributions / loans, vote on own
Treasurer	Invite (NO), Confirm contributions, Confirm-and-disburse loans, Record repayments	Vote on own loan
Secretary	Invite	Confirm anything
Member	Apply for a loan, contribute, vote on others' loans	(no admin actions)

The "Apply for a loan" dialog includes a **live projection** mirroring the server's interest math, plus a **pot-balance gate** so the borrower can't submit a principal that exceeds available pot.

After voting, the Approve/Reject buttons disappear and a "You voted: Approve" line shows in primary colour — this is driven by the server's `my_vote` field on the loan response.

Token / session lifecycle

- **Access token TTL** is 15 minutes (`JwtSettings.AccessTokenExpiryMinutes`,

configurable per-tenant). Refresh token TTL is 7 days.

- `TokenRefresher` keeps the access token fresh on every gRPC call.
- A "session lock" screen (`SessionLockScreen.kt`) kicks in after a

configurable inactivity period (default 5 min) — requires PIN to resume, doesn't drop the session. Biometric unlock if enrolled.

- On `sessionManager.logout()` the access/refresh tokens + customer_id +

account_id are cleared but **phone_number is retained** so the next launch lands on the PIN-only login. `fullLogout()` clears everything.

Build configuration

`mobile/androidApp/build.gradle.kts` has the relevant flags:

Flag	Debug	Release
GRPC_HOST	10.0.2.2 (emulator → host loopback)	api.goldbank.co.zw
GRPC_PORT	5000	443
GRPC_USE_TLS	false (h2c)	true
DEFAULT_TENANT_ID	goldbank	goldbank

10.0.2.2 is the magic IP that, from inside an Android emulator, hits the host machine's localhost. On a physical device on the same Wi-Fi, override with the host's LAN IP.

DI

shared/.../di/AndroidDataModule.kt registers everything in a single Koin single module:

```
single { SecureStorage(get()) }            // EncryptedSharedPreferences
single { SessionManager(get(), tenantId) }
single {                                    // gRPC channel — once per app
    GrpcChannelFactory(
        host = grpcHost, port = grpcPort, useTls = useTls,
        interceptors = listOf(get<AuthClientInterceptor>(), get<RetryInterceptor>()),
    )
}
single<ManagedChannel> { get<GrpcChannelFactory>().create() }
single { AccountGrpcClient(get()) }
…
single { TokenRefresher(
    sessionManager = get(), authRepository = get(),
    deviceIdProvider = { Settings.Secure.getString(...) }
) }
```

ViewModels go in PresentationModule:

```
viewModel { EkubViewModel(get(), get()) }
viewModel { AssetViewModel(get(), get(), get()) }
…
```

Common gotchas

- **The Kotlin Language Server in VS Code** sometimes shows

"Unresolved reference" red squigglies on freshly-created files in the same package. Gradle resolves them fine. If a build mysteriously fails with "classes.jar in use by another process", kill the LSP process (fwcd.kotlin) and re-run.

- grpcCall { … } **is a top-level suspend function**. Don't call

blocking code inside — switch threads with withContext(Dispatchers.IO) if needed. The auto-refresh check itself runs on the calling dispatcher.

- **Adaptive icon on API 26+** uses @drawable/splash_logo as the

foreground; legacy bitmaps for pre-API-26 are regenerated to match. Launcher caches icons — adb shell pm clear com.google.android.apps.nexuslauncher to force a refresh after icon changes.

- **Customer ID vs Account ID**: most APIs are customer-scoped now.

Always pass sessionManager.getCustomerId() for asset / Ekub calls; use getAccountId() for currency-bound things like balance / transfer source.

- **Device transfer**: re-binding an account to a new phone is a

separate flow (InitiateDeviceTransfer + OTP). authenticate will refuse a new device with Auth.DeviceMismatch until the transfer is approved.

Testing

There are no unit tests on mobile yet. Smoke testing is `adb`:

```
# Build, install, launch
cd c:\Users\wmapu\Projects\GoldBank\mobile
.\gradlew :androidApp:assembleDebug
$adb = "$env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe"
& $adb install -r androidApp\build\outputs\apk\debug\androidApp-debug.apk
& $adb shell am force-stop com.goldbank.app
& $adb shell monkey -p com.goldbank.app -c android.intent.category.LAUNCHER 1

# Watch refresh logs
& $adb logcat -s TokenRefresher
```

Demo PIN for every seeded user is `1234`. Demo phones:

Role	Phone
Chairman (Borrowdale Savings)	`+263770003287` (Tendai Moyo)
Treasurer	`+263775304489` (Chiedza Mutasa)
Secretary	`+263771882741` (Farai Chikwanha)
Member	`+263774538185` (Nyasha Dube)
Registered (gold coins)	`+263771000001` (John Moyo)

Bank-client (Back-office Admin Web)

A React + Vite + Material UI single-page app for back-office staff (admins, KYC officers, fraud analysts, compliance, loan officers, support, branch managers). Runs on the Vite dev server at **http://localhost:5173/** and proxies `/api/*` to the gateway on `localhost:5001`.

```
bank-client/
  index.html
  vite.config.js          ← :5173, /api proxy → :5001
  package.json
  src/
    App.jsx               ← route table + role-gated ProtectedRoute
    auth/
      AuthContext.jsx     ← dev-stub login against SEED_ACCOUNTS
      roles.js            ← Roles enum + Access matrix + SEED_ACCOUNTS
      ProtectedRoute.jsx  ← role guard wrapper
    services/
      api.js              ← every fetch call against /api/admin/*
      snackbar.jsx        ← global toast provider
    pages/
      Dashboard.jsx
      Customers.jsx
      KycReview.jsx
      Disputes.jsx
      FraudAlerts.jsx
      Transactions.jsx
      UserManagement.jsx
      BranchManagement.jsx
      DepositHouses.jsx
      LoanReview.jsx
      AssetValuation.jsx
      SystemConfig.jsx
      Merchants.jsx
      Tariffs.jsx
      AuditTrail.jsx
      UserGrowthReport.jsx
      MerchantReport.jsx
      RevenueReport.jsx
      ReconReport.jsx
      Login.jsx
      NotFound.jsx
    layouts/
      MainLayout.jsx       ← top bar, side nav, user menu
    theme.js               ← MUI theme (dark/light toggle)
```

Authentication

Currently a **dev stub** — there's no server-side admin login flow yet. `auth/roles.js` hard-codes seven seed accounts:

Username	Password	Role
`admin`	`Admin@1234`	Admin
`kyc`	`Kyc@1234`	KycOfficer
`fraud`	`Fraud@1234`	FraudAnalyst
`support`	`Support@1234`	CustomerService
`loans`	`Loans@1234`	LoanOfficer
`compliance`	`Compliance@1234`	ComplianceOfficer
`branch`	`Branch@1234`	BranchManager

`AuthContext.login()` checks these locally and stores the user in sessionStorage. **No tokens, no server roundtrip.** The role string is the basis for in-app guards but does NOT secure the underlying REST endpoints — `AdminApiController` is currently unauthenticated. Wiring this to real JWTs (e.g. against `admin_users` + `TellerAuthController`'s pattern) is the obvious next-step debt; see the standing offer in `server.md`.

Role-based access

`Access` matrix in `roles.js`:

```
KycAccess:        [Admin, KycOfficer, BranchManager]
FraudAccess:      [Admin, FraudAnalyst, BranchManager]
CustomerAccess:   [Admin, CustomerService, BranchManager]
DisputeAccess:    [Admin, CustomerService, BranchManager]
LoanAccess:       [Admin, LoanOfficer, BranchManager]
ReportAccess:     [Admin, ComplianceOfficer, BranchManager]
UserManagement:   [Admin, BranchManager]
ConfigAccess:     [Admin]
```

Routes in `App.jsx` are wrapped with `<ProtectedRoute roles={Access.XAccess}>…</ProtectedRoute>`. If the logged-in role isn't in the array, the page renders an Alert: *"You do not have permission to view this page."* Side-nav items use the same matrix to hide links — UI/UX consistency at the cost of duplicated configuration.

Page reference

Dashboard (`/`)

Read from `/api/admin/dashboard` — live counters, 30-day transactions chart. Visible to all logged-in roles.

Customers (`/customers`) — CustomerAccess

- Search by name / phone / email, filter by status (`active`,

`suspended`, `frozen`, `closed`).

- Row click → detail dialog with all-currency balances, KYC level,

document timeline (`/api/admin/customers/{shortId}/activity`).

- Actions: Activate / Suspend / Freeze / Unfreeze / Reset PIN / Close

→ `POST /api/admin/customers/{shortId}/actions { action, reason }`.

KYC Review (`/kyc`) — KycAccess

Pending KYC documents with AI extraction sidebar. Approve / reject / escalate. Reads `kyc_documents` + `kyc_verifications`.

Disputes (`/disputes`) — DisputeAccess

Filter by status (Open, Investigating, Resolved). Detail panel shows the underlying transaction, customer history, activity log. Actions update `disputes.status` + `resolution` + `refund_amount`.

Fraud Alerts (`/fraud`) — FraudAccess

Sorted by severity. Detail panel shows the rule that triggered + a human-readable AI explanation (calls `AiService.ExplainFraudAlert`). Status workflow: `New` → `Reviewed` → (`Escalated` | `Dismissed`).

Transactions (`/transactions`)

All transactions, filterable by type, status, accountId. No role gate — visible to anyone logged in (the page is purely read-only).

User Management (`/users`) — UserManagement

List of `admin_users`. Create / update / deactivate.

- `GET /api/admin/admin-users`
- `POST /api/admin/admin-users` (create)
- `PUT /api/admin/admin-users/{id}` (update)

Role + branch dropdowns drive the JWT claims those staff get when they log in to bank-teller.

Branch Management (`/branches`) — UserManagement

CRUD for `branches`. Each teller / branch-manager is bound to one branch via `admin_users.branch_id`.

Deposit Houses (`/deposit-houses`) — ConfigAccess

The trusted vault facilities that hold customer assets. Name, address, license number, **trust status** (`Verified`, `Probationary`, `Suspended`), API endpoint (for automated valuation sync — not wired yet).

Loan Review (`/loans`) — LoanAccess

Pending and active loans. Loan officers approve/reject; can view collateral assets linked via `loans.collateral_asset_ids`.

Asset Custody & Valuation (`/assets`) — LoanAccess

Three tabs:

1 **Asset Registry** — every asset in custody, filterable by

type / status, click row for detail.

1 **Valuation Queue** — assets with `lastValued > 30 days ago`,

sorted by days overdue. Per-row "Assign valuer" and "Value" buttons. The "Value" dialog submits to `POST /api/admin/asset-valuations` which writes a row to `asset_valuations` and bumps `assets.LastValuationAmount`.

1 **Valuation History** — every recorded valuation with prev → new +

% change.

System Config (`/config`) — ConfigAccess

Key-value editor for `system_configs`. Categorised UI: PIN policy, OTP, fraud thresholds, daily / monthly limits, Ekub monthly fees, AI model selection.

Merchants (`/merchants`) — CustomerAccess

Merchant onboarding queue (KYB documents), active merchants list, suspend / activate actions.

Tariffs (`/tariffs`) — ConfigAccess

Per-tenant fee configuration (`tenant_fee_configs`). Transfer fees, bill-pay fees, agent commissions, FX margin.

Audit Trail (`/audit`) — ReportAccess

Pageable view of `audit_logs`. Filter by admin user, action type, date range. Read-only.

Reports — ReportAccess

Route	Source
`/reports/users`	`ReportingService.GetUserGrowth`
`/reports/merchants`	`ReportingService.GetMerchantReport`
`/reports/revenue`	`ReportingService.GetRevenueReport`
`/reports/recon`	`ReportingService.GetReconciliationReport`

Each report has a CSV export button calling `ReportingService.Export` streaming.

REST endpoint map (bank-client → gateway)

Every page hits `/api/admin/*` via `services/api.js`:

Function	Endpoint	Page
`getDashboard`	`GET /api/admin/dashboard`	Dashboard
`generateCustomers`	`GET /api/admin/customers`	Customers
`generateCustomerActivity`	`GET /api/admin/customers/{shortId}/activity`	Customer detail dialog
`postCustomerAction`	`POST /api/admin/customers/{shortId}/actions`	Customer actions
`generateTransactions`	`GET /api/admin/transactions`	Transactions
`getDisputeActivities`	`GET /api/admin/disputes/{shortId}/activities`	Dispute detail
`generateAdminUsers`	`GET /api/admin/admin-users`	User Management
`createAdminUser`	`POST /api/admin/admin-users`	User Management
`updateAdminUser`	`PUT /api/admin/admin-users/{id}`	User Management
`generateBranches`	`GET /api/admin/branches`	Branch Management
`generateAssets`	`GET /api/admin/assets`	Asset Custody, Asset Registry tab
`generateAssetValuations`	`GET /api/admin/asset-valuations`	Asset Custody, History tab
`submitAssetValuation`	`POST /api/admin/asset-valuations`	Asset Custody, Value dialog

Plus `KycReview`, `FraudAlerts`, `Disputes`, `LoanReview`, `Merchants`, `DepositHouses`, `SystemConfig`, `Tariffs`, `AuditTrail`, and the four report pages each have their own pair of endpoints — see `bank-client/src/services/api.js` for the full list.

Running locally

```
cd c:\Users\wmapu\Projects\GoldBank\bank-client
$env:OPENSSL_CONF = ''   # clear the broken system OpenSSL config if present
npm install              # first time
npm run dev              # serves on http://localhost:5173/
```

The Vite proxy in `vite.config.js` forwards `/api/*` to `http://localhost:5001`, which is the gateway's REST port. Vite's hot reload picks up `.jsx` changes within seconds.

If the gateway is down: every page shows an empty / error state and the browser console has `[api] /<path> failed: TypeError: fetch failed`. Fix the gateway, refresh.

Conventions

- **No global store.** State is local to each page (`useState` + `useEffect`). Don't introduce Redux/Zustand without a strong reason — the current pages have shallow state.
- **MUI components only.** No bespoke CSS-in-JS unless absolutely necessary. The `theme.js` overrides cover the gold-on-navy palette.
- **Always handle** `null` from `fetchApi(...)` — it's the convention for any API failure (logs to console, returns `null`). Pages render an "Unable to load" state in that case.
- **Currency strings**: `Number(value).toLocaleString(undefined, { minimumFractionDigits: 2 })` — never use raw JS arithmetic on money beyond that.
- **Routes are URL-typed** (`/customers/:shortId`, not query params). Keep deep-linkable.
- **Role gating is UI-only.** The REST API does not currently enforce it. **Don't ship to prod without** wiring `[Authorize(Roles="...")]` on `AdminApiController` and replacing the SEED_ACCOUNTS stub.

Known limitations

- **No real authentication.** SEED_ACCOUNTS is a dev stub. The AdminApiController is unauthenticated.
- **No CSRF protection.** Required once real auth is added.
- **Customer search is server-side filtering on** `Contains` — fine for the demo's 20 customers, will scan-table on a real dataset. Add a trigram index on `(first_name, last_name, phone, email)` for prod.
- **CSV export streams chunks** but the React side buffers the whole response before triggering a download. For huge reports, switch to `ReadableStream` + `showSaveFilePicker`.
- **Branch supervisor view** isn't yet a separate page — branch managers use the standard customer list + filter manually by branch.

- **Reports don't paginate** — they emit one big dataset that gets

rendered with `DataGrid`'s built-in pagination. Above ~10k rows this gets sluggish.

Bank-teller (Branch Counter Web)

A React + Vite + Material UI web app for **branch tellers**, **branch managers**, and **vault managers**. Runs at **http://localhost:5174/**. Unlike bank-client, this one **does** authenticate against the gateway — it's the most "real" web surface today.

```
bank-teller/
  vite.config.js          ← port 5174
  src/
    App.jsx               ← route tree with HomeRouter (role-based)
    auth/
      TellerSessionContext.jsx  ← real JWT session
      ProtectedRoute.jsx
    components/
      DenominationGrid.jsx ← cash-count UI for shift open/close
      ErrorBoundary.jsx
      SecurityShell.jsx    ← inactivity lock screen
    layouts/MainLayout.jsx
    services/api.js
    pages/
      Login.jsx
      Dashboard.jsx
      SupervisorDashboard.jsx
      VaultDashboard.jsx
      CustomerSearch.jsx
      CustomerCard.jsx
      Deposit.jsx
      Withdrawal.jsx
      Drawer.jsx           ← shift open / close
```

Authentication

Real JWT, server-side validated. `POST /api/teller/auth/login` with `{ Username, Password }` returns `{ accessToken, user }`. The accessToken is a JWT with claims: `sub` (admin_user.Id), `role` (Teller/BranchManager/VaultManager/Admin), `tenant_id`, `branch_id`, `username`.

The `Authorization: Bearer <jwt>` header rides on every API call. On `401` the session is cleared and the user bounced to `/login`. There's no refresh token currently — when the JWT expires, the user logs back in. (See the standing offer to add refresh; mobile already has it via `TokenRefresher`.)

Auth wrapping flow:

```
TellerSessionProvider          ← React Context
  ↓
ProtectedRoute                 ← redirects to /login if no token
  ↓
SecurityShell                  ← inactivity timer → lock screen
  ↓
MainLayout                     ← top bar, side nav, logout
  ↓
HomeRouter                     ← BranchManager/Admin see SupervisorDashboard,
                                 everyone else sees Dashboard
```

Roles

Demo credentials (PINs not used here — username/password instead):

Username	Password	Role	What they see
`teller`	`teller`	Teller	Dashboard + Customer card + Deposit + Withdrawal + Drawer
`branch`	`branch`	BranchManager	Supervisor Dashboard + everything tellers see + Vault
`admin`	`admin`	Admin	Everything

Passwords come from the `DemoSeeder` (username == password for demo). Real deployments would use BCrypt hashes seeded externally.

Page reference

Login (`/login`)

Username + password form. Calls `loginTeller(username, password)`. On success stores `(accessToken, user)` in sessionStorage and navigates to `/`. If the response role is `Admin` or `BranchManager`, they land on the supervisor dashboard.

Dashboard (`/`) — Teller view

- Open shift CTA if no active drawer.
- KPIs for today: deposits / withdrawals count + value, transactions

reversed, fees collected.

- Quick links to Customer Search, Deposit, Withdrawal.

Supervisor Dashboard (`/`) — BranchManager / Admin view

- Branch-wide KPIs (active tellers, open drawers, total cash).
- Per-teller cards showing shift status + cash position.
- High-value pending approvals (transactions above the daily limit

that need supervisor sign-off).

Vault Dashboard (`/vault`) — BranchManager / VaultManager

- Branch vault denomination breakdown (notes + coins per currency).
- Movement log (cash in / out with denominations).
- "New movement" dialog (in/out, amount, denominations, reference,

witness).

- Spot-check workflow: take an inventory count, compare with ledger,

adjust if variance.

Customer Search (`/customers`)

- Free-text search across phone / national ID / card PAN / name.
- Server-side via `GET /api/teller/customers/search?q=...`.
- Row click → `/customers/:accountId`.

Customer Card (`/customers/:accountId`)

Three tabs:

1 **Profile** — Personal info, balances per currency, KYC status,

flags (Frozen / Suspended / SignatureVerified).

1 **Transactions** — Date-ranged transaction history. From/To pickers,

results table.

1 **Assets** — Custom-built for the gold-custody product. Lists every

asset the customer owns, each row showing description, type, quantity, receipt #, deposit house, last valuation, status, and a `Collateral` chip if the asset is securing an open loan.

The Assets tab has two action paths:

Deposit asset

"Deposit asset" button → dialog with:

- Deposit house picker (loaded from `GET /api/teller/deposit-houses`)
- Receipt number
- Asset type (GoldCoin / GoldBar / Silver / Platinum / PreciousStone /

Other)

- Description
- Quantity + unit
- Weight (grams) + purity (0–1)
- Optional initial valuation + currency

On submit → `POST /api/teller/customers/{accountId}/assets`. Server validates uniqueness of `(deposit_house, receipt_number)`, creates the asset in `PendingVerification` status, writes the initial valuation row if provided.

Withdraw asset

Per-row "Withdraw" button (disabled when `isCollateral` or non-Active). Confirms via a small dialog with an optional reason. On submit → `POST /api/teller/assets/{assetUuid}/withdraw`.

Critical pre-flight check: the server pulls every non-Closed / non-Defaulted loan across **all of the customer's sibling accounts** and filters in-memory (EF can't translate `Contains` on the JSON `collateral_asset_ids` column). If any loan lists this asset as collateral, the response is **409 Conflict** with:

```
{
  "error": "Cannot withdraw — asset is collateral on an open loan.",
  "blockingLoan": {
    "loanId": "LOAN-043521",
    "reference": "LOAN-COLL-TEST",
    "outstanding": 1000.00,
    "currency": "USD",
    "status": "active"
  }
}
```

The dialog surfaces this inline — the teller sees the loan reference + outstanding balance and can advise the customer to settle the loan first. On 200, the asset is soft-deleted (`is_deleted = true`, `status = Released`, `deleted_by = tellerId`) and disappears from the list on refresh.

Deposit (`/deposit`)

Cash-deposit workflow:

1 Select customer (search-as-you-type).

2 Enter amount + currency.

3 Denomination breakdown (which notes/coins the customer handed

over) — validated client-side and again server-side.

1 Reference / narration.

2 Submit → creates a `Transaction` + `BranchCashTransaction`,

credits the account balance.

1 Print receipt (PDF stream from `/api/teller/transactions/{id}/receipt.pdf`).

Above-limit deposits route to a pending-approval queue visible to the branch manager.

Withdrawal (`/withdrawal`)

Mirror of Deposit:

1 Select customer.

2 Verify identity (ID type + number + signature shown alongside DB

record).

1 Enter amount + currency.

2 Denomination breakdown (which notes you handed out).

3 Pull from drawer / from vault if drawer is short.

4 Submit → debits account, deducts from drawer.

Drawer (`/drawer`)

Shift management:

- **Open**: Count starting cash with denominations grid → record. The

shift session is created; all subsequent transactions are attributed to it.

- **Close**: Count ending cash. The system computes

expected = *opening* + *deposits* – *withdrawals* – *fees* + *adjustments* and shows the variance. Submit → seals the shift, generates EOD report PDF (`/api/teller/drawer/{id}/eod-report.pdf`).

If variance exceeds tolerance the close is gated on supervisor approval.

REST endpoint map

Function	Endpoint	Page
`loginTeller`	`POST /api/teller/auth/login`	Login
`getDashboard`	`GET /api/teller/dashboard`	Dashboard
`searchCustomers`	`GET /api/teller/customers/search`	Customer Search
`getCustomerCard`	`GET /api/teller/customers/{accountId}/card`	Customer Card → Profile tab
`getCustomerTransactions`	`GET /api/teller/customers/{accountId}/transactions`	Customer Card → Transactions tab
`listCustomerAssets`	`GET /api/teller/customers/{accountId}/assets`	Customer Card → Assets tab
`registerCustomerAsset`	`POST /api/teller/customers/{accountId}/assets`	Asset Deposit dialog
`withdrawAsset`	`POST /api/teller/assets/{assetId}/withdraw`	Asset Withdraw dialog
`listDepositHouses`	`GET /api/teller/deposit-houses`	Deposit dialog picker

Function	Endpoint	Page
`getCurrentDrawer`	`GET /api/teller/drawer/current`	Dashboard / Drawer
`openDrawer`	`POST /api/teller/drawer/open`	Drawer
`closeDrawer`	`POST /api/teller/drawer/close`	Drawer
`postDeposit`	`POST /api/teller/deposits`	Deposit
`postWithdrawal`	`POST /api/teller/withdrawals`	Withdrawal
`approveWithdrawal`	`POST /api/teller/withdrawals/{pendingId}/approve`	Supervisor Dashboard
`reverseTransaction`	`POST /api/teller/transactions/{cashTxnId}/reverse`	Customer Card / Supervisor
`getBranchDashboard`	`GET /api/teller/branches/{branchId}/dashboard`	Supervisor Dashboard
`getVault`	`GET /api/teller/vaults/{vaultId}`	Vault Dashboard
`postVaultMovement`	`POST /api/teller/vaults/{vaultId}/movements`	Vault Dashboard
`postSpotCheck`	`POST /api/teller/vaults/{vaultId}/spot-checks`	Vault Dashboard

All of the above (except `auth/login`) require a valid teller JWT and return 401 otherwise. They additionally enforce **tenant gates**: `account.TenantId == tellerJwt.tenant_id` (the string `"goldbank"` for the demo).

Running locally

```
cd c:\Users\wmapu\Projects\GoldBank\bank-teller
$env:OPENSSL_CONF = ''
npm install
npm run dev   # http://localhost:5174/
```

The Vite config does not configure a /api proxy — REST calls go to `http://localhost:5001` directly via the `API_BASE` constant in `services/api.js`. This means **CORS must be enabled** on the gateway's REST host, which it is via `[EnableCors("BankClient")]` on the controllers.

Workflows

A customer comes in to deposit gold coins

1 Customer Search → enter phone (`+263770003287`).

2 Open customer card → Assets tab.

3 Click "Deposit asset". Fill the dialog (deposit house = GoldVault

Harare, receipt # = GV-2026-0106, asset type = GoldCoin, description, qty = 1, unit = coins, weight = 31.10, purity = 0.999, initial valuation = 2800, currency = USD).

1 Submit. The asset appears in the table immediately with status

`PendingVerification` and the bank's valuer will promote it to `Active` from the bank-client UI.

A customer comes in to withdraw their gold coins

1 Find the customer's card → Assets tab.

2 Identify the row by receipt #. The Withdraw button:

- **Disabled** with tooltip if `isCollateral: true` → "Cannot

withdraw — collateral on LOAN-XYZ". Ask the customer to settle the loan first.

- **Disabled** if status ≠ Active (e.g. already `PendingRelease`).

- **Enabled** otherwise.

1. Click Withdraw → optional reason → Confirm.
2. The physical handover happens at the counter; the row vanishes from

the table. The asset row is preserved soft-deleted in the DB for audit.

Closing a shift with a small cash variance

1. Drawer → Close shift.
2. Count cash by denomination, enter actuals.
3. System computes `actual − expected` per denomination.
4. If total variance is within tolerance (configurable in

`system_configs`), the close completes and an EOD PDF is generated.

1. If outside tolerance, the close is stalled pending supervisor

approval. The supervisor sees the pending close on their dashboard.

Conventions

- **Tenant** is always `"goldbank"` for demo. The JWT carries it; every

endpoint that touches customer data checks it.

- **Identity verification at the counter**: signature image, ID number,

and DOB are shown side-by-side on the customer card. The `signatureVerified` flag tracks whether a teller has confirmed the signature in person — it's a manual click + reason field.

- **PDF receipts** stream from the gateway via `ReceiptPdfService`

(QuestPDF). Print directly from the browser tab that opens.

- **High-value transactions** require supervisor approval. Defined by

`fraud.high_value_threshold_zwg` and `fraud.high_value_threshold_usd` in `system_configs`.

- **Inactivity lock** kicks in after 5 minutes (configurable). PIN-less

unlock for now (just resume with username + password).

Operations

How to run the system locally, where the moving parts live, and how to troubleshoot when something goes wrong.

Quick start

```
# 1. Start the container backbone (Postgres, Redis, gateway, switch, admin, etc.)
cd c:\Users\wmapu\Projects\GoldBank
python -m podman_compose --profile core up -d

# 2. Apply migrations + seed demo data (first time only)
cd server\GoldBank.Migrator
$env:GOLDBANK_ConnectionStrings__DefaultConnection = `
  "Host=localhost;Port=5432;Database=goldbank;Username=goldbank;Password=goldbank_dev_password"
dotnet run -- --demo
cd ..\..
```

```
# 3. Start the React dev servers (in two separate terminals)
cd bank-client; $env:OPENSSL_CONF = ''; npm run dev      # :5173
cd bank-teller; $env:OPENSSL_CONF = ''; npm run dev      # :5174

# 4. Build + install the mobile APK (emulator must already be running)
cd mobile
.\gradlew :androidApp:assembleDebug
$adb = "$env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe"
& $adb install -r androidApp\build\outputs\apk\debug\androidApp-debug.apk
```

Container layout

`docker-compose.yml` defines all the long-running services. Three profiles select which ones come up:

Profile	Brings up
`core` *(default for dev)*	postgres, redis, gateway, switch, admin, hsm, notifications, server-migrator, switch-migrator
`ai`	ollama (multi-GB image; start separately when AI features needed)
`monitoring`	prometheus, grafana, elasticsearch, kibana

To bring up a single service:

```
python -m podman_compose --profile core up -d gateway
```

Port map

Service	Container port	Host port	Protocol
postgres	5432	5432	PostgreSQL
redis	6379	6379	Redis
gateway (gRPC)	1111	**5000**	gRPC h2c (plaintext HTTP/2)
gateway (REST)	1112	**5001**	HTTP/1.1
switch (gRPC)	3333	3333	gRPC h2c
switch (web admin)	8080	8080	HTTP
admin (Blazor)	5010	5010	HTTP
hsm	5005	5005	HTTP
notifications	5007	5007	HTTP
ollama	11434	11434	HTTP (OpenAI-compatible)
bank-client (Vite dev)	—	**5173**	host process
bank-teller (Vite dev)	—	**5174**	host process
mobile (Android emulator)	—	uses `10.0.2.2:5000`	host loopback

Environment variables

The compose file reads from `.env` (not committed) and falls back to defaults. The `.env.example` at the repo root lists every variable. Common overrides:

Variable	Default	Purpose
`POSTGRES_USER`	`goldbank`	DB user
`POSTGRES_PASSWORD`	`goldbank_dev_password`	DB password
`POSTGRES_DB`	`goldbank`	DB name
`ASPNETCORE_ENVIRONMENT`	`Development`	.NET env switch

Variable	Default	Purpose
GATEWAY_GRPC_PORT	5000	host gRPC port
GATEWAY_HTTP_PORT	5001	host REST port
JWT_SECRET	dev-secret-key-change-in-production-min-32-chars!!	HMAC key. **Rotate for prod.**
JWT_ISSUER	goldbank-dev	JWT issuer claim
JWT_AUDIENCE	goldbank-api	JWT audience claim

Seeding

The `GoldBank.Migrator` console app is the migration runner **and** the demo seeder. CLI flags:

```
dotnet run                                  apply all migrations (PublicDb + GoldBankDb)
dotnet run -- --context PublicDb            PublicDbContext only
dotnet run -- --context GoldBankDb          GoldBankDbContext only
dotnet run -- --demo                        migrations + seed demo data
dotnet run -- --apply-ekub-fees             debit the Ekub monthly fee against active groups
dotnet run -- --apply-ekub-fees --period 2026-05    apply fee for a specific period
```

The seed is **idempotent** — re-running it does nothing if rows already exist. To wipe + reseed, drop the DB:

```
podman exec goldbank-postgres psql -U goldbank -d postgres -c `
  "DROP DATABASE IF EXISTS goldbank; CREATE DATABASE goldbank;"
cd server\GoldBank.Migrator
dotnet run -- --demo
```

Demo accounts

Role	Username / phone	Password / PIN
Back-office admin (bank-client)	admin	Admin@1234
KYC officer	kyc	Kyc@1234
Fraud analyst	fraud	Fraud@1234
Customer support	support	Support@1234
Loan officer	loans	Loans@1234
Compliance	compliance	Compliance@1234
Branch manager	branch	Branch@1234
Branch teller (bank-teller)	teller	teller
Branch manager (bank-teller)	branch	branch
Customer (chairman)	+263770003287	1234
Customer (treasurer)	+263775304489	1234
Customer (secretary)	+263771882741	1234
Customer (member)	+263774538185	1234
Customer (member, has gold coins)	+263771000001	1234

Auxiliary seeds (SQL scripts)

The demo seeder runs in C# but a few demo bits live in raw SQL under `scripts/`. These are one-off — run them via:

```
Get-Content scripts\<name>.sql | podman exec -i goldbank-postgres `
  psql -U goldbank -d goldbank
```

Script	What it does
`seed-gold-coins.sql`	Inserts 1 deposit house + 5 gold-coin assets for John Moyo (`+263771000001`)
`seed-asset-valuations.sql`	Inserts 10 prior valuations (initial + revalued) for the 5 coins so the History tab shows % change
`age-asset-valuations.sql`	Backdates `last_valuation_date` on 2 assets so the Valuation Queue has overdue items
`check-ekub-mig.sql`	Diagnostic dump of Ekub group state (memberships, roles, contributions, pot)

Scheduled jobs

Ekub monthly fee

The bank's per-product fee on active Ekub groups. Idempotent: at most one `ekub_fees` row per `(group, period)`.

Manual run:

```
cd server\GoldBank.Migrator
dotnet run -- --apply-ekub-fees
```

Scheduled run (Windows): register a Task Scheduler entry to run on the 1st of every month at 02:00 local time:

```
.\scripts\register-ekub-fees-task.ps1            # register
.\scripts\register-ekub-fees-task.ps1 -RunNow    # register + trigger immediately
.\scripts\register-ekub-fees-task.ps1 -Unregister # remove
```

The task invokes `dotnet run --project server/GoldBank.Migrator -- --apply-ekub-fees` against the local DB. Because it has to reach `localhost:5432`, this can't be a cloud-hosted scheduled agent; it has to run on the host that has the DB.

Diagnostics

Container status

```
podman ps                                          # running
podman ps -a --format "{{.Names}} | {{.Status}}"   # all
podman logs goldbank-gateway --tail 50             # gateway logs
podman logs goldbank-postgres --tail 20            # DB logs
```

Health endpoints

Service	Health
Gateway	container has a `HEALTHCHECK` running `dotnet --info`. Status visible in `podman ps`.
Postgres	`podman exec goldbank-postgres pg_isready -U goldbank`
Redis	`podman exec goldbank-redis redis-cli ping`

Common failure modes

Symptom	Likely cause	Fix
`Cannot connect to Podman socket: 127.0.0.1:5346`	Podman machine VM is down even though `podman machine list` says running	`wsl --shutdown` then `podman machine start`
`bind: address already in use` on 6379 (or other)	Orphaned `wslrelay.exe` holding the port	`Get-NetTCPConnection -LocalPort 6379` → find PID → `Stop-Process -Id <pid> -Force`
`HTTP_1_1_REQUIRED` / `Received Goaway` from mobile or web	Gateway container has exited; clients hit nothing on :5000/:5001	Check `podman ps`; restart container
Mobile build fails on `classes.jar in use by another process`	VS Code Kotlin LSP (`fwcd.kotlin`) holds the file	`Get-CimInstance Win32_Process -Filter "Name='java.exe'"` → find one with `kotlin*Server` → kill it
`Vite` won't start, `Could not determine Node.js install directory`	Bash shell quirk on Windows	Use `pwsh` instead, or `$env:OPENSSL_CONF=''` before `npm run dev`
`BC8D0000:error:07000065:configuration file routines:def_load_bio:missing equal sign`	A broken system-wide `OPENSSL_CONF` env var points at a non-OpenSSL file	`$env:OPENSSL_CONF = ''` in the shell before any node command
Bank-client gets 403 on a customer's data	The customer's `tenant_id` doesn't match the staff JWT tenant	`UPDATE bank.customers SET tenant_id='goldbank' WHERE phone='+263…'` (also for accounts)
`Insufficient pot balance` when applying for an Ekub loan	Server-side pre-flight rejection (correct)	Check `GetGroupDetail.potBalance` vs principal; deduct in-flight pending loans
Confirm-and-disburse silently fails on the mobile group detail screen	Error not surfaced; status response code was 9 (FailedPrecondition)	Look at logcat; fixed in mobile by surfacing `state.error` as a banner

Logs

The gateway uses Serilog with structured logs to stdout. To follow:

```
podman logs goldbank-gateway --follow
```

Adjust verbosity with `Logging__LogLevel__Default` env var (default `Information`). For SQL traces add `Logging__LogLevel__Microsoft.EntityFrameworkCore=Information`.

Backups (operational, not configured for dev)

Postgres data lives in a podman volume:

```
podman volume inspect GoldBank_postgres-data
```

For prod the recommended pattern is `pg_dump` on a schedule + S3 upload. The repo doesn't currently include a backup container.

Building images

```
# Gateway only (most common)
cd c:\Users\wmapu\Projects\GoldBank
podman build -f server\GoldBank.Gateway\Dockerfile -t localhost/goldbank_gateway:latest .

# Switch
podman build -f switch\Containerfile -t localhost/goldbank_switch:latest ./switch
```

```
# Admin (Blazor)
podman build -f admin\GoldBank.Admin\Dockerfile -t localhost/goldbank_admin:latest .

# Notifications
podman build -f server\GoldBank.Notifications\Dockerfile -t localhost/goldbank_notifications:latest .
```

After rebuilding an image, restart the container so it picks up the new image:

```
podman stop goldbank-gateway
podman rm goldbank-gateway
python -m podman_compose --profile core up -d gateway
```

For frontend changes (bank-client, bank-teller), Vite hot-reloads — no rebuild needed.

For mobile changes:

```
cd mobile
.\gradlew :androidApp:assembleDebug
# install + relaunch
$adb = "$env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe"
& $adb install -r androidApp\build\outputs\apk\debug\androidApp-debug.apk
& $adb shell am force-stop com.goldbank.app
& $adb shell monkey -p com.goldbank.app -c android.intent.category.LAUNCHER 1
```

CI

A `Jenkinsfile` and a `.gitlab-ci.yml` exist at the root but neither is wired to a runner in this checkout. Both target a standard build-test-package pipeline against .NET 10. CI work is pending.

Production deployment

Not in scope of this document — the repo is dev-tuned. To deploy:

1. Replace `JWT_SECRET` with a real 32+ char secret.
2. Set `Postgres` credentials to non-default values; rotate.
3. Terminate TLS at a load balancer; expose gateway gRPC over TLS or

private network only.

1. Replace bank-client's `SEED_ACCOUNTS` stub with a real login flow.
2. Add real authentication to `AdminApiController` (`[Authorize]` +

admin JWT issuer).

1. Configure Ollama with a production-class model and GPU.
2. Replace the in-DB KYC document storage with S3/MinIO.
3. Wire CI to build images + push to a registry.

These are tracked as standing offers in the system docs but none are done in the repo as of `4be4dd7`.